



Design Justification File

Issue 1.1

Table of Contents

1 Introduction	6
1.1 Objective	6
1.2 Scope	6
1.3 Applicable Documents	6
1.4 Reference Documents	7
1.5 Reference Standards	7
1.6 Definitions	8
2 Architecture	9
3 Sensor selection	10
4 Mechanical design	11
5 Structural and thermal analysis	14
5.1 Overall Design	14
5.2 Sine, Random & Shock Constraints	14
5.3 Thermal Constraints	16
6 Operational ranges analysis	17
7 Synthetic data generation	20
8 Algorithm selection	21
8.1 Experiment campaign plan	21
8.2 Object detection model selection	27
8.3 Model training	29

8.4 Model testing and analysis	37
8.5 Deep learning frameworks trade-off	39
8.6 Catalog of models	41
8.7 Embedding results	42
8.8 Sensor fusion trade-off	44
9 Simulator updates	47
9.1 Inertial measurement unit (IMU) model	47
10 GNC flight software	49
10.1 Navigation filters	49
11 Functional testing facility	52
12 Functional testing satellite model	54

Date created: 02/12/2024

Distribution:

- LMO (internal),
- European Space Agency (ESA),
- Luxembourg Space Agency (LSA)

Change Log:

- 1.0: First release
- 1.1: Update for SDR
 - Added [Sensor selection](#)
 - Added [Embedding results](#)
 - Added reference of DJF from previous phase
 - Non-operating temperature for SIRC added

Position**Name****Signature****Date****Prepared by**

System Engineer

David Regad

Reviewed by

FPGA Application Engineer

Lino Alves

Approved by

System Engineer

David Regad

Released by

Program Manager

Cedric van den Meerssche

Luxembourg Export Control Rating: Not Applicable

This document is of Luxembourg origin and is Copyright © of LMO Sarl. and has been generated by LMO under ESA contract 4000136351/21/NL/VR - Distribution Limited as per Contract Conditions.

This document is for Internal Use within the Luxembourg National Space Programme LuxIMPULSE only

1 Introduction

LMO in partnership with SnT (University of Luxembourg) are carrying out the Development of In-Orbit Servicing Space Situational Awareness payloads using deep learning (DIOSSA) in the framework of an ESA project (contract No. 4000136351/21/NL/VR). The RPO system is a suite of advanced vision-based navigation technologies for autonomous rendezvous and proximity operations, using deep learning computer vision algorithms on edge. It autonomously derives the 6 degrees-of-freedom pose estimation of a target space resident object under any illumination condition and is part of the spacecraft GNC system. The payload is composed of several monocular cameras, the control and vision processing electronics, and the software. The objective of this project is the design, development, verification and qualification of the RPO system for space application.

1.1 Objective

The objective of this document is to present the rationale, parameters, assumptions and models used to develop the DIOSSA RPO system.

1.2 Scope

This document is applicable to the CCN1 project.

The description and outcomes of the previous phase of the project can be found in the corresponding DJF, see [AD-2].

1.3 Applicable Documents

AD-n	Document Title	Issue
AD-1	DP-0001 - Synthetic data package	1.0

AD-2

PL21-0002-DJF-0001 - System Design Justification File

2.1

1.4 Reference Documents

RD-n	Document Title	Issue
RD-1	Fayyad, Jamil, et al. "Deep learning sensor fusion for autonomous vehicle perception and localization: A review."	Sensors 20.15 (2020): 4220
RD-2	Proceedings of SPIE, "Environmental evaluation of the ULIS PICO1024 microbolometer, by Thierry Dartois et. al.	
RD-3	Park, et al. "Robust multi-task learning and online refinement for spacecraft pose estimation across domain gap"	Advances in Space Research (2024): Volume 73, Issue 11

1.5 Reference Standards

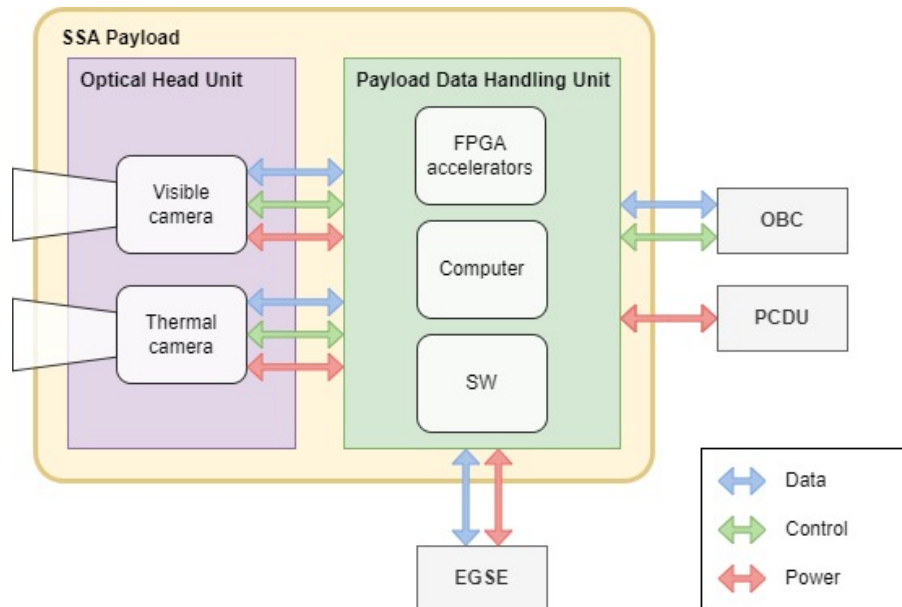
RS-n	Document Title	Issue
RS-1		

1.6 Definitions

Acronym	Meaning
MLI	Multi layer insulation
OHU	Optical Head unit
OTS	Off the shelf
PTQ	Post training quantization
QAT	Quantization aware training
TIR	Thermal Infrared Camera
VIS	Visible spectrum camera

2 Architecture

In this phase, the architecture of the design remains the same as in previous phase.



RPO system architecture diagram

3 Sensor selection

The visible spectrum camera has been selected in the first phase of the DIOSSA project.

The thermal infrared camera from MDA has been selected based on the following aspects:

- Detector selection and performance: MDA camera uses the PICO1024 microbolometer version (originally developed by ULIS, currently Lynred) which has already gone through an extensive qualification campaign conducted with ESA (see [RD-2]). The camera features a 17um pixel pitch which, as far as LMO current knowledge goes at the moment of writing this, has the best performance in the market for space capable microbolometers, both in terms of sensitivity, resolution and robustness.
- Maturity: MDA has produced already flight models of the camera, confirming a maturity higher than TRL7 and MDA capacity to deliver such payload. Although not much flight heritage data is available at the moment, it is expected to become available shortly.

Reliability and Quality assurance aspects have not been taken into consideration yet in this selection, because LMO hasn't reached that level of engagement with the supplier at this stage of the project.

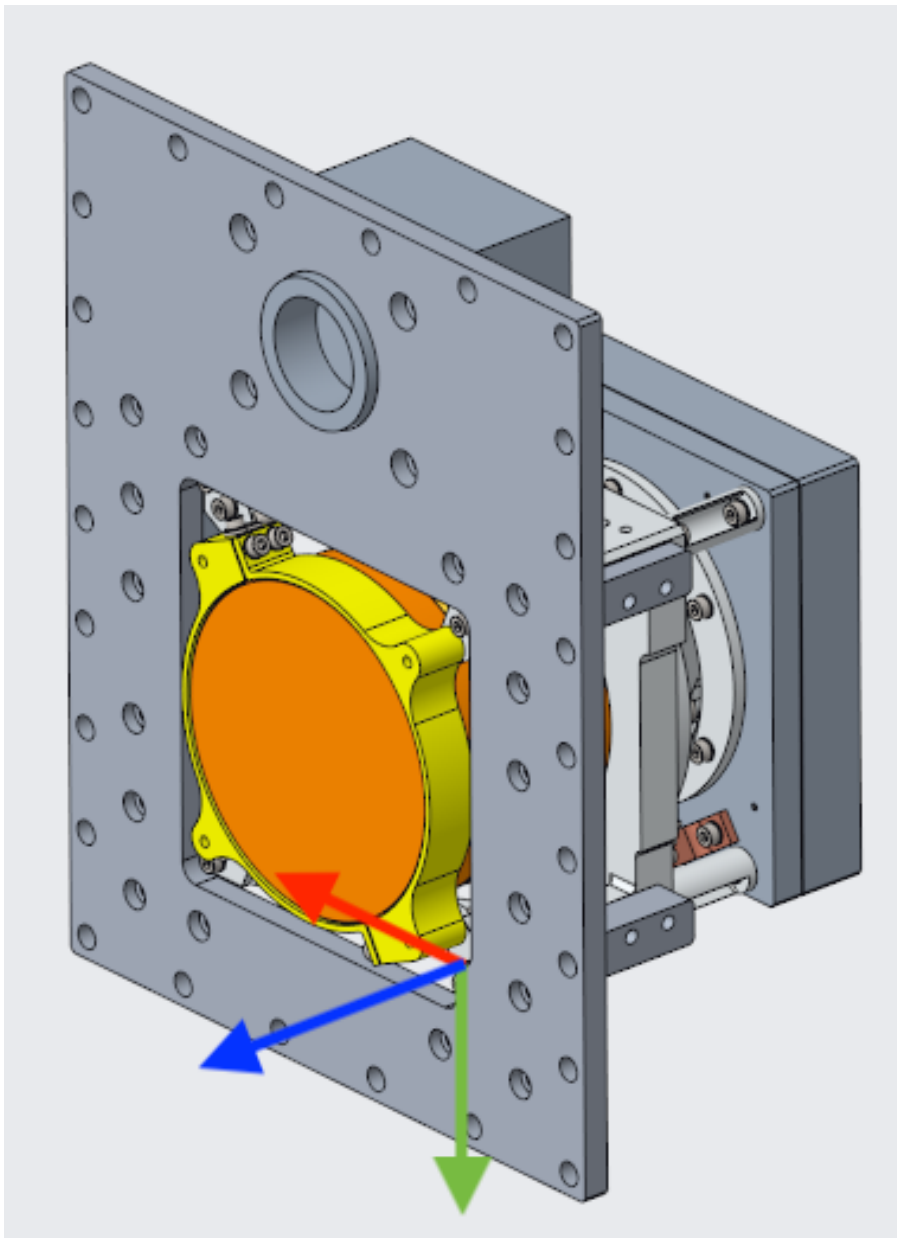
The statements of compliance for the 2 cameras (VIS and TIR) are provided in annex.

4 Mechanical design

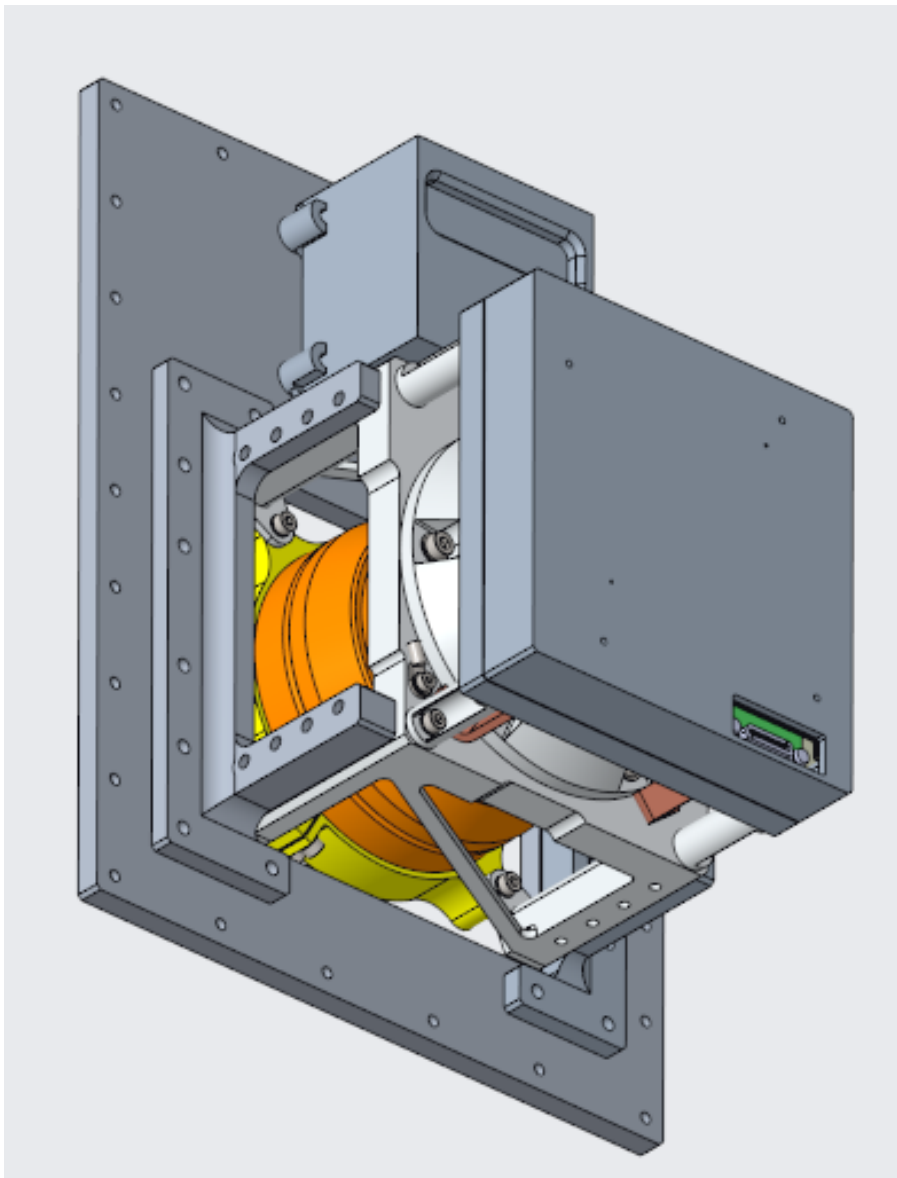
As opposed to previous design, LMO and ESA have agreed to have a new design with the computer separated from the optics.

As the computer is procured with its enclosure, only the Optical Head Unit (OHU) needs to be designed, with the following assumptions:

- the baseline is 1 VIS and 1 TIR camera
- the structure serves both as mechanical and thermal interface
- the interface between the OHU structure and the spacecraft structure must be on the same side as the interfaces between the cameras and the OHU structure (corresponding to the camera openings)
- the structure includes a reference point visible when the OHU is mounted on the spacecraft that can be used for alignment measurement



Front side of the OHU assembly with reference frame (X: red; Y: green; Z: blue)



Back side of the OHU assembly

5 Structural and thermal analysis

As part of the DIOSSA Payload, LMO has incorporated the SpectraCAM visual camera from Redwire and the Infrared SIRC camera from MDA. For the electronics the baseline is that this will be mounted together with the rest of the platform electronics and not as part of the payload optical head. They will therefore not be considered as part of this assessment.

As there are not yet clear defined platform requirements from an environmental perspective for the GEA RISE or ENCORE Mission LMO will define the constraints of the DIOSSA Payload based on the qualification environments of the key components of its system which are the two cameras.

The following chapter reviews the mechanical and thermal constraints of this design.

5.1 Overall Design

The current design expects the sensors to be integrated via a frame in which the cameras are mounted. The frame design will directly connect to the satellite structure with no secondary structures to mount the cameras on. The frame is baselined to be designed in aluminium.

Based on this the frame will have no impact on the thermal or structural behaviour of the payload. It is therefore assumed that the interface of the cameras with the frame can be identical to them being interfaced with the platform.

This means that the DIOSSA payload requirements will be set based on the constraints defined by each camera. In future LMO will work with the customer to ensure that either the customer meets the dedicated environmental constraints of the Payload (i.e. the cameras) or that a specific design/delta-qualification is done.

5.2 Sine, Random & Shock Constraints

Based on inputs from the suppliers LMO defined Random & Shock requirements which would encompass the level of both cameras. These result in:

Random (All Axes)

Hz

g²/Hz

20	0.026
----	-------

50	0.16
----	------

800	0.16
-----	------

2000	0.026
------	-------

Shock (All Axes)

Hz	g
----	---

100	30
-----	----

1000	1500
------	------

10000	1500
-------	------

For Sine it was identified that the current qualification inputs provided for the SpectraCAM are not yet available so for now only those related to the SIRC are provided.

Sine (All Axes)

Hz	g
----	---

5	0.55
---	------

18	7.17
----	------

20	8.85
----	------

21	9.76
----	------

26.5	15.54
------	-------

30	20
----	----

60	20
----	----

60	6
----	---

100	6
-----	---

5.3 Thermal Constraints

With respect to thermal constraints, LMO similarly reviewed the qualification references of the SIRC & SpectraCAM and identified the Maximum & Minimum Operating temperature envelope for both camera which is:

- Operating temperature: -10 to 40°C

Here are the non-operating temperature ranges for the 2 cameras:

- SpectraCAM: **-34 to 71°C**
- SIRC: -36 to 75 °C

6 Operational ranges analysis

The analysis of the different conditions the system could see and their corner cases led us to refine the different parameters to take into account for development and validation.

Due to selected architecture and the nature of the two deep neural network models - first object detection, then keypoint regression - some cases cannot be addressed with the current technology and should be identified by the system to avoid sending wrong information to the AOCS system.

As it was chosen to handle the two sensors separately to allow operating only with one of them, the analysis also needs to be done separately.

The two tables below list the conditions that might cause issues to the system, the associated ranges, the stage where the issue would be identified and the output the RPO system should send to the AOCS system.

Operational ranges with VIS camera

Condition	Range	Object Detection	Keypoint Regression	Out-of-range flag
Sun phase angle	0-160 deg	OK	OK	No
Sun in FoV	160 - 180 deg	NOK	NOK	Yes
Eclipse / No object in FoV	Illum. less than 50%	NOK	NOK	Yes
Stars in FoV	N/A	OK	OK	No
Earth in FoV	N/A	OK	OK	No
Moon in FoV	[0 - 100] % illuminated	OK	OK	No

Stray light	N/A	OK	OK	No
Solar array outside FoV	[0 - 100) % visible	OK	OK	No
Body outside FoV	[0 - 100) % visible	OK	NOK	Yes
Distance out of range	Nominal: 20 - 100 m	OK	NOK	Yes

Operational ranges with TIR camera

Condition	Range	Object Detection	Keypoint Regression	Out-of-range flag
Thermal control active	0 - +60°C	OK	OK	No
Thermal control inactive	-100 - +200°C	OK	OK	No
Sun in FoV	160 - 180 deg	NOK	NOK	Yes
No sat	N/A	NOK	NOK	Yes
Stars in FoV	N/A	OK	OK	No
Earth in FoV	N/A	OK	OK	No
Moon in FoV	[0 - 100] % illuminated	OK	OK	No
Stray light	N/A	OK	OK	No

Solar outside FoV	array	[0 - 100) % visible	OK	OK	No
----------------------	-------	---------------------	----	----	----

Body outside FoV		[0 - 100) % visible	OK	NOK	Yes
------------------	--	---------------------	----	------------	------------

Distance range	out of	Nominal: 20 - 100 m	OK	NOK	Yes
-------------------	--------	------------------------	----	------------	------------

7 Synthetic data generation

The synthetic images were generated in PANGU, according to the mission specifications, i.e. target satellite, operational range, orbital approach and sensor characteristics. The major change compared to DIOSSA phase 1 is that the target satellite is Inmarsat-5, orbiting in GEO (instead of OneWeb in LEO). Another change is the candidate approach trajectories, which now start from a distance of 100m with a flyaround at 20m and end at 2m.

In light of this information, we could define the boundary conditions for the synthetic data generation. For the training dataset, we generated 32400 images, randomly sampled from 225 attitudes on the Fibonacci lattice, from 12 distances from 12 Gaussian distributions and from 12 sun positions, again following the Fibonacci lattice. The validation dataset was generated based on the trajectories provided by the GNC subsystem and were not used for training. There were 7 candidate approach trajectories, each with approximately 7000 image, a total of 48635 images.

Regarding the satellite modelling, the 3D model designed by LMO, based on publicly available information of the satellite, such as the dimensions, the solar panels, photographs and materials. We have created a process of importing a satellite's CAD model into PANGU and using that model to generate synthetic images (more details in the Synthetic Data Package [AD-1] documentation).

The modelling of the camera in PANGU followed the specifications of the actual camera used in the lab campaign, with the same resolution and intrinsic parameters as defined by the calibration process.

8 Algorithm selection

8.1 Experiment campaign plan

An *experiment* in this context refers to training, inference and validation of a given task model with all of its hyperparameters. A *task model* refers to a machine learning model that solves a particular task, for example: object detection or keypoint regression. Furthermore, hyperparameters include but are not limited to: model selection (FCOS, FasterRCNN, etc), learning rate and scheduler, batch size, loss function.

8.1.1 Task models

In the context of DIOSSA CCN phase, we have 2 task models:

- **Object Detection** - a model that finds a predefined object in the field of view and outputs its location in form of a rectangular bounding box (aligned to the horizontal line) and a prediction score in the domain $[0, 1]$
- **Keypoint Regression** - a model that finds n predefined keypoints in the image that correspond to n 3d points on the given 3-D model (eg: a satellite). Not all n keypoints might be in the field of view and not all might be in direct line of sight (some might be occluded by the object itself). The model outputs n 2d locations (x,y) coordinates and n $M \times M$ 2d heatmaps that correspond to the probability density of the corresponding keypoints.

8.1.2 Metrics

We have fixed the following metrics for the task models:

- **Object detection:**
 - **Primary:** IoU (Intersection-over-Union)
 - **Secondary:** mAP (mean average precision, tightly correlated in this project due to 1 object in the scene)
- **Keypoint Regression:**
 - **Primary:** L1 error on all keypoints

- **Support:** Average score
- **Secondary Support:** Average covariance components (xx,xy,yy)

8.1.3 Hyper-parameter search space

Hyper-parameters are parameters that are not optimized during the training itself, that is during back-propagation. They are instead set before training begins and remain the same for the duration of the training run. These include for example image input size, number of keypoints, learning rate scheduling strategy, etc.

8.1.4 Transfer learning

Transfer learning refers to using the knowledge learned by a neural network on a similar task to boost the performance, significantly reduce training time and required data on your desired task. For example, for satellite object detection we can apply transfer learning by using an object detection model that was trained on the COCO dataset, which contains 80 different objects. Here are transfer learning strategies, we are considering:

- **Object detection:**

- Start training from scratch (randomly initialized weights, no transfer learning)
- Start training from the backbone pretrained on ImageNet or similar, where the rest is initialized to random weights
- Start training from COCO dataset

- **Keypoint regression:**

- Start training from scratch (randomly initialized weights, no transfer learning)
- Start training from the backbone pretrained on ImageNet or similar, where the rest is initialized to random weights
- Start training from HumanPose dataset (predicts locations of human joints to estimate the human pose)
- Start training from one of the best performing models used in DIOSSA-1

8.1.5 Static image processing

We refer to *static image processing* by all the image processing that is always applied to image and does not change from an image to image. That includes image cropping and/or padding,

resizing, undistorting, autocropping, etc. It does **not** include any data augmentation, which would only be used during training.

The search space of all possible center crops and resizes is of course infinite and choosing too many possible scenarios will lead to a combinatorial explosion of number of experiments needed to run. Therefore, we have decided that we will proceed only with 3 rough size categories as the task model input sizes:

Keypoint Regression:

- large [448x448]
- medium [224x224]
- small [112x112]

Object Detection:

- large [640x480]
- medium [384x288]
- small [224x224]

Note

Object detection task model

The object detection task model would suffer from cropping too much. Therefore the model will accept a rectangular image that deviates as little as possible from the raw camera image in terms of aspect ratio and is still a valid input size (some architectures have requirements on aspect ratios, height and width)

Note

Keypoint regression task model

The keypoint regression model produces heatmaps that are proportional to the input image size. For MobilePose, that ratio is 1/4, ie for a 224x224 input image the output heatmap size is 56x56. Larger heatmap sizes should in theory provide better accuracy but it remains to be seen. Furthermore, this model is trained on ideal object locations meaning that we crop the image to remove a lot of unnecessary information and avoid losing signal during downsizing. Assuming the object detection model is a perfect detector, the expansion ratio (the padding added to the bounding box) added before cropping can be minimal or zero. Therefore, we can start with a fixed expansion ratio of 5-10 %, but decrease it if we find that the object detection

model performs very well in order to increase the accuracy of the keypoint regression task model

Things that still need considering:

- Pixel depth: 8, 12, 16 or other ? Does it have an impact if we are quantizing the model to 8bit precision.
- Number of channels; Would it make sense to train on gray-scale images ?

8.1.6 Data augmentation

Data augmentation is a technique that should lead to a better generalization of the task model. Its goal is to disturb images by adding different kinds of noise in order to learn a better internal representation of the images that will lead to a better generalization across the domain gap (synthetic to lab to space). At least that's our hypothesis. It has proven itself empirically to be a valuable tool in achieving a greater performance on a variety of machine learning tasks.

We will follow the data augmentation approach from D'Amico paper [RD-3] and might need to revise the approach if the performance is still lacking. However, since we are not trying to close the domain gap in DIOSSA-CCN1, we have decided not to use data augmentation in this model training iteration.

8.1.7 Other typical hyperparameters

The remaining typical (but not less important!) hyperparameters are: - Optimizer (Adam, RMSProp, SGD, etc) - Learning rate - Learning rate schedules - L2/drop-out regularization - Number of epochs - Loss function specifics:

- MobilePose has a sigma parameter for JS heatmap loss that can be changed
- Constrain loss function to work only on visible keypoints
- Loss function components can have weighting factors
- Batch size, usually set to highest possible that still fits inside your GPU

Some of the above parameters have a massive impact on the stability of the training, eg learning rate and some have less. Finding them is usually done through trial-and-error process or doing a grid/random search. There exist some method on performing a more guided search through the hyperparameter space, though it is still immature and largely academic.

8.1.8 The general approach

- We train and evaluate each task model separately; no end-to-end pose estimation evaluation at this stage
- We train only on synthetic data, evaluation on synthetic and on lab data
- **From all the possible hyper-parameters, we should classify them as first and second class citizens:**
 - **First class citizens: We experiment first with those that we expect to have the highest impact. After the first round of experiments we freeze them and continue with the rest in a second round. These could be:**
 - Learning from scratch vs Transfer learning
 - No data augmentation vs various data augmentation strategies
 - Learning rate and scheduling
 - Static image processing (image sizes: small/medium/large)
 - **Second class citizens: We target to fine-tune an already well designed experiment from the previous round. For example:**
 - Train with or without L2 regularization and/or dropout
 - Train using all keypoints vs using only visible keypoints
 - Train with all 20 keypoints or a subset (maybe 10?)
 - Train with different loss functions and/or weighting
 - Training epochs

8.1.9 Note on quantization

Quantization is necessary for deployment to edge devices, which do not have floating point operations. We refer to these models as *quantized*, whereas *float* models are models trained and evaluated on the GPU/CPU, which **cannot** be directly run on the edge device.

There are 2 types of quantization. Post Training Quantization is easier, but usually does not provide results as good as QAT.

- **Post Training Quantization (PTQ)**
 - PTQ maps the traditional FP32 activation space to a reduced INT8 space. This mapping has to be calibrated with your representative sample dataset in order to limit the information loss incurred during the mapping.

- **Quantization Aware Training (QAT)**

- Quantization Aware Training is a technique that constraints your weights to be already quantized during training, it should therefore offer highest accuracy on the edge model. However, it usually requires a decent float32 model, which is used as a starting point for the QAT. We only apply QAT on our best performing candidate models.

8.1.10 Datasets

We have 2 datasets available:

1. Random positions and orientation dataset
2. Approach trajectory datasets

The 1) random dataset is split randomly following a standard 60-20-20 rule meaning 60% for training, 20% for validation, 20% for testing. Models are exclusively trained on the training split, and compared against each other on the validation split. The remaining test split is only used at the final stage, when the best candidates are evaluated in order to reduce data leakage as much as possible.

The final models are also evaluated on the trajectory datasets. We have 7 trajectories with different illumination conditions and split into two approach lengths [2; 100] meters and [2; 300] meters.

8.1.11 Future work: Additional experiments

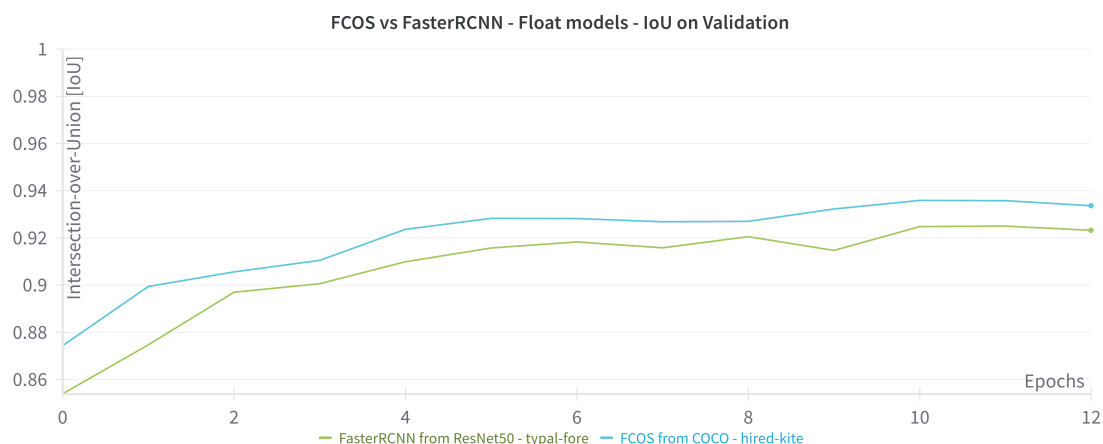
We plan additional experiments that will give us more understanding of the problem at hand:

- Understand the amount of data required for a training set. Essentially find out when any new additional training images do not increase performance. This can be achieved by running experiments where validation set stays the same, but training set is limited by x samples where x is a number from a list of exponentially increasing numbers from 50 to 100_000 (or training image size). If the performance does not plateau then we might have to increase the number of training images.
- Train on non-optimal inputs (ie the output of the object detection, should increase robustness of the keypoint regression model)
- **Differentiate between visibility categories for the keypoint regression network and penalize depending on the category**
 - 1. Visible: in direct of line of sight
 - 1. Occluded: occluded by the object itself

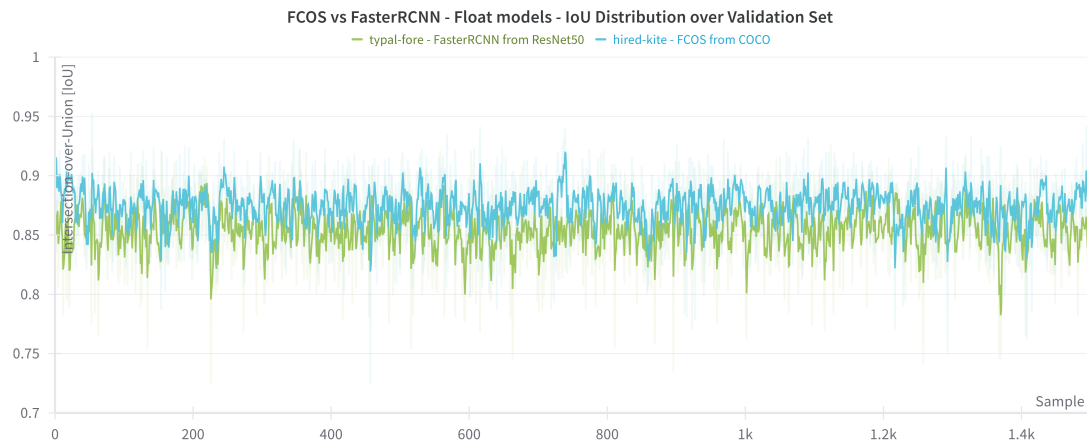
- -1) Out of field-of-view: when keypoints were cropped out

8.2 Object detection model selection

The model selection for the object detection task was performed on two most promising models, namely FasterRCNN and FCOS. The first one is a two-stage detector composed of a Region Proposal Network (RPN) followed by ROI Pooling operation and finally terminated with regression and classification heads. On the other hand, FCOS is a single-stage detector with a single neural network based on a Feature Pyramid Network (FPN) acting as the network backbone. It treats object detection as a per-pixel regression and classification task making it a much simpler architecture. Typically, FasterRCNN is attributed with higher accuracy (especially on smaller objects) on but also higher latency compared to FCOS, it also takes longer to train the network. Another important limitation of FasterRCNN is that it is not possible to do Quantization Aware Training, because of it's architecture. Nonetheless, we have performed various experiments to test the hypothetical higher performance of FasterRCNN compared to FCOS and we actually arrived at opposites results.



Best training for both models. FCOS consistently outperforms FasterRCNN by approximately 0.02 IoU



Distribution of IoU on the validation set. FCOS consistently outperforms FasterRCNN on almost all samples.

Comparison of FCOS and FasterRCNN float models

Object Detection Model	FCOS	FasterRCNN
Training time (12 epochs)	5h	11.3h
Number of parameters	32.1M	43M
Input image size [H,W]	480 x 640	480 x 640
IoU on validation set	0.874	0.853
mAP @ 75 IoU on validation set	0.956	0.901
Latency on RTX 3070 Laptop GPU	45.78 FPS	37.66 FPS

As shown on the two plots above and the table, FCOS is a clear winner in this object model selection. It's faster, more accurate, takes less time to train, has lower memory footprint and is an eligible candidate for QAT.

8.3 Model training

We trained each model separately and evaluate its performance individually. The best performing float model is used as the starting point for QAT so that we can accurately determine the drop in performance due to quantization. We also only show good experiments, meaning that we dropped the experiments that crashed (eg: because the batch size was too high) or did not converge due to poor hyper-parameter choice, such as extreme learning rates or ineffective learning rate scheduler settings.

8.3.1 Object Detection model training

The table below illustrates 8 final experiments. There are 4 groups here. Networks trained from scratch, where weights were initialized randomly (*model_cfg/pretrained/weights* is empty) or we applied transfer learning from a model trained on COCO dataset. It can be seen that all experiments without transfer learning achieve subpar performance when compared with experiments with the same settings but initialized with COCO weights. We also tested the influence of the input image size on the accuracy of the network. As expected, a bigger image input size leads to a higher performance. For example, when we compare **saved-aril** with **licit-weal** , the performance drop on IoU is 0.033 and around 0.04 on mAP @ IoU>075, which could be an acceptable drop in performance given a speed up close to 50% (65 instead of 46 FPS).

Object Detection float model training experiments

Run Name:	gamey-tote	licit-weal	natal-oats	ledgy-quay	shill-rues	saved-aril	unwet-judo	minim-prow
Duration:	14.0h	14.1h	4.8h	5.0h	11.4h	11.3h	3.5h	N/A
dataloader/ batch_size	20	20	20	20	4	4	4	4
model_cfg/ model_kwargs/ img_size	[384, 288]	[384, 288]	[384, 288]	[384, 288]	[640, 480]	[640, 480]	[640, 480]	[640, 480]

Run Name:	gamey-tote	licit-weal	natal-oats	ledgy-quay	shill-rues	saved-aril	unwet-judo	minim-prow
model_cfg/ pretrained/ weights		COCO		COCO		COCO	COCO	COCO
train_cfg/ lr_scheduler/ parameters/ gamma	0.8	0.8	0.5	0.5	0.5	0.5	0.5	0.5
train_cfg/ trainer/trainer/ max_epochs	81	81	32	32	32	32	10	55
Completed epochs	81	81	32	32	32	32	10	33
Bounding box centerness loss	0.611	0.608	0.614	0.609	0.593	0.588	0.589	0.588
Bounding box regression loss	0.106	0.074	0.13	0.082	0.103	0.042	0.055	0.043
Bounding box classification loss	0.128	0.167	0.093	0.124	0.149	0.077	0.076	0.081
Total loss	0.845	0.849	0.837	0.814	0.845	0.707	0.72	0.711
IoU epoch	0.893	0.925	0.87	0.918	0.898	0.958	0.944	0.957
mAP	0.784	0.858	0.739	0.845	0.805	0.939	0.906	0.938

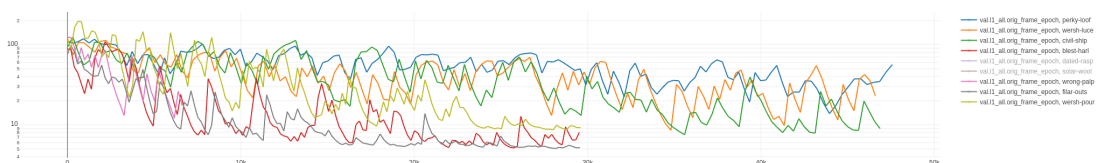
Run Name:	gamey-tote	licit-weal	natal-oats	ledgy-quay	shill-rues	saved-aril	unwet-judo	minim-prow
mAP @ IoU>50	0.989	0.989	0.988	0.989	0.99	0.99	0.99	0.99
mAP @ IoU>75	0.906	0.946	0.867	0.947	0.938	0.988	0.986	0.988
Inference speed @ Laptop RTX 3070 [fps]	65.12	65.12	65.12	65.12	46.13	46.13	46.13	46.13

8.3.2 Keypoint regression model training

Next, we trained the Keypoint Regression network. In the same fashion, we tested the impact of transfer learning as well as different input image size. We also compared models with different number of predicted keypoints. The first observation is that models trained longer perform better. Secondly, transfer learning has a huge impact on convergence and stability of the training process as also illustrated on the plot below. Finally, we have not found that the higher image input size [448x448] compared to [224x224] has a positive impact on performance. Instead, it has increased the training time significantly. Furthermore, we used two keypoints sets and found that the model with all 41 keypoints performs significantly worse than those trained on only a subset. Our best model **blest-harl** achieves 3.799 L2 pixel error and it was trained starting from a model from DIOSSA-1.

Keypoint Regression float model training experiments									
Run Name:	perky-loof	wersh-luce	civil-ship	blest-harl	dated-rasp	solar-wool	wrong-palp	filar-outs	wersh-pour
Duration:	13.3h	13.0h	13.1h	8.3h	8.9h	2.1h	1.9h	8.3h	8.3h
dataloader/ batch_size	8	8	8	32	32	32	32	32	32
model_cfg/ model_in- put_shape	[3, 448, 448]	[3, 448, 448]	[3, 448, 448]	[3, 224, 224]	[3, 224, 224]	[3, 224, 224]	[3, 224, 224]	[3, 224, 224]	[3, 224, 224]
model_cfg/ model_kwarg/ keypoints	20	20	20	20	20	20	20	20	41
model_cfg/pre- trained/ weights		DIOSSA-1 @ PanguV2	HumanPose	DIOSSA-1 @ PanguV2		HumanPose	DIOSSA-1 @ PanguV2	HumanPose	HumanPose
net/train- able_params	4102272	4102272	4102272	4102272	4102272	4102272	4102272	4102272	4102944
train_cfg/ trainer/ trainer/ max_epochs	150	150	150	150	150	32	32	150	150
Coordinate loss	0.018	0.013	0.014	0.01	0.013	0.014	0.014	0.011	0.012
Heatmap loss	0.068	0.051	0.054	0.016	0.025	0.028	0.028	0.017	0.018
Total loss	0.085	0.064	0.068	0.026	0.038	0.043	0.042	0.028	0.03
L1 keypoint error on input image [2590 x 1942] [px]	7.82	6.042	6.434	4.783	6.55	6.538	6.555	4.988	8.467
L1 keypoint error on model input size [px]	5	3.784	3.967	1.407	1.89	2.042	2.034	1.495	1.65
L2 keypoint error on input image [2590 x 1942] [px]	6.181	4.782	5.091	3.799	5.2	5.186	5.202 3.96		7.275
L2 keypoint error on model input size [px]	3.958	2.994	3.142	1.114	1.495	1.615	1.608	1.181	1.308
Inference speed @ Laptop RTX 3070 [fps]	229.17	229.17	229.17	235.08	235.08	235.08	235.08	235.08	235.08

Keypoint Regression float model training experiments



Keypoint Regression Float model - performance over training time

8.3.3 Object detection QAT model training

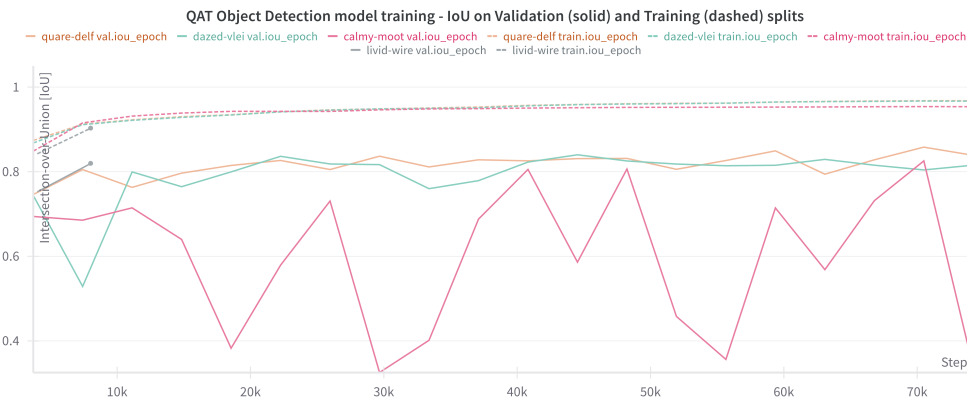
In the QAT phase of model training, we take the best performing float model and use it as the starting point. We can start training from scratch or from a model trained on a different dataset such as HumanPose, but we did not find that it produces a model with sufficient accuracy. Here, we ran 4 experiment. Experiment **livid-wire** was trained on the bigger image input size [640x480] and we were not able to train it sufficiently long due to a higher memory requirement for QAT. We

were force to use batch size of 2, which is generally not sufficient to provide enough stability during training, because the gradient becomes too volatile. Yet, we were able to achieve 0.819 IoU on the validation set, which means a performance drop of 0.139 (down from saved-aril's 0.958 IoU). Furthermore, we ran the same experiment on the smaller image size [384x288], which allowed us to raise the batch size to 7 (from 2) and train the model for much longer without it crashing (11.5 hours instead of 47.7min). Our best model **quare-delf** achieved 0.858 IoU, which means a performance drop of only 0.067 (down from licit-weal's 0.925). We also ran the same experiment with different learning rates and quantizer learning rate and observed that QAT is even more susceptible to those parameters.

Object Detection QAT model training experiments

Run Name:	quare-delf	dazed-vlei	calmy-moot	livid-wire
Duration:	11.4h	11.6h	11.5h	47.7min
dataloader/batch_size	7	7	7	2
dataloader/limit				4000
model_cfg/model_kwargs/ img_size	[384, 288]	[384,288]	[384, 288]	[640, 480]
model_cfg/pretrained/weights	licit-weal-best	licit-weal-best	licit-weal-best	saved-aril-best
train_cfg/lr_scheduler/ parameters/gamma	0.8	0.8	0.8	0.5
train_cfg/optimizer/parameters/ lr	0.001	0.001	1.00E-05	0.0005
train_cfg/qat/quantizer_lr	0.01	0.01	0.01	
train_cfg/trainer/trainer/ max_epochs	20	20	20	2

Run Name:	quare-delf	dazed-vlei	calmy-moot	livid-wire
Bounding box centerness loss	0.611	0.61	0.615	0.6
Bounding box regression loss	0.148	0.161	0.186	0.161
Bounding box classification loss	0.31	0.127	0.652	3.12
Total loss	1.069	0.898	1.452	3.881
IoU epoch	0.858	0.84	0.825	0.819
mAP	0.693	0.661	0.627	0.658
mAP @ IoU > 50	0.978	0.979	0.976	0.944
mAP @ IoU > 75	0.85	0.819	0.74	0.826
mAP for large obj (area > 9216 px)	0.746	0.7	0.687	0.67
mAP for medium obj (1024 < area <= 9216 px)	0.691	0.658	0.617	0.653
mAP for small obj (area < 1024px)	0.623	0.666	0.605	-1



Object Detection Quantization Aware Training (QAT) - performance over training time

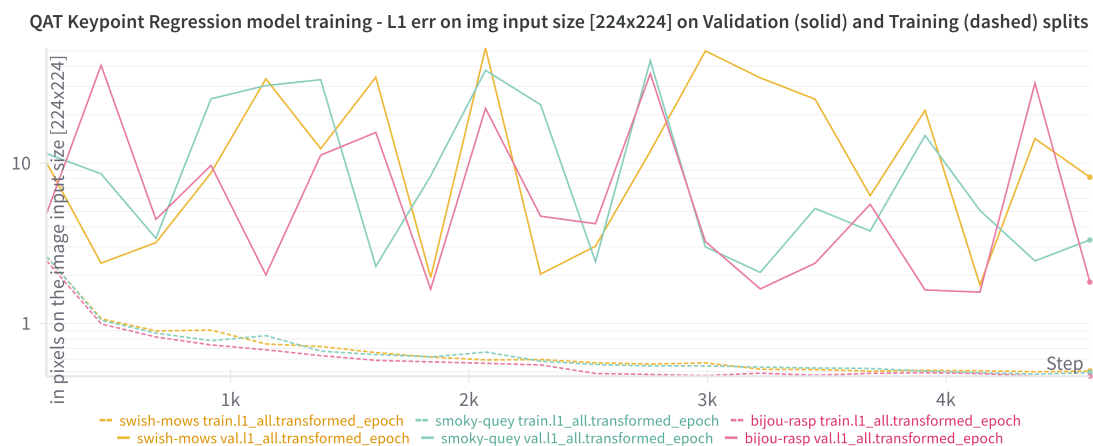
8.3.4 Keypoint regression QAT model training

For Keypoint Regression we selected **blest-harl** as the starting point. We also found that the learning rate is a tricky parameter to set and we recommend running multiple experiments with various learning rates and quantizer learning rates. Our best model **bijou-rasp** achieved 5.303 L1 error [px] on the original image [2590x1942] compared with 4.783, which means that the performance drop due to quantization is around 0.520. The plot below also illustrates that higher learning rate improves stability of the training process meaning that the performance between epochs does differ by a large margin.

Keypoint Regression QAT model training experiments

Run Name:	bijou-rasp	smoky-quey	swish-mows
Duration:	1.6h	1.6h	1.6h
model_cfg/pretrained/weights	blest-harl-best	blest-harl-best	blest-harl-best
train_cfg/optimizer/parameters/lr	1.00E-05	1.00E-06	1.00E-07
train_cfg/qat/quantizer_lr	0.01	0.01	0.01
train_cfg/trainer/trainer/max_epochs	20	20	20

Run Name:	bijou-rasp	smoky-quey	swish-mows
Coordinate Loss	0.011	0.015	0.012
Heatmap Loss	0.022	0.029	0.024
Total Loss	0.033	0.044	0.036
L1 keypoint error on input image [2590 x 1942] [px]	5.303	6.848	5.797
L1 keypoint error on model input size [px]	1.564	2.076	1.74
L2 keypoint error on input image [2590 x 1942] [px]	4.216	5.451	4.613
L2 keypoint error on model input size [px]	1.238	1.646	1.379



Keypoint Regression Quantization Aware Training (QAT) - performance over training time

8.4 Model testing and analysis

Having the best float and quantized models trained on their respective tasks, we can proceed to final model testing and analysis on the pose estimation task. We evaluated pairs of models on the remaining test split of the random training dataset as well as 7 realistic approach trajectories. The metric we use to evaluate the end-to-end performance is the relative position error [%], rotational error [degrees] and the number of valid frames. We achieve sub-2% relative position error with 1-sigma on float model on trajectories 1-3 and 4-5, and very good performance on trajectory 4 and 7. On the QAT model evaluated on the edge device (3rd row of each table), we achieve sub-2% relative error on trajectories 2 and 3. Trajectory 1 shows problems with the rotation error that is very close to 180, this is due to the fact that the chosen keypoints have too much symmetry, which leads to correct position prediction (on the float models) and 180 deg rotation error.

Relative Position Error [%] - 1 sigma (68.27%): Range restricted to [20; 100] [m]

		trajectory01	trajectory02	trajectory03	trajectory04	trajectory05	trajectory06	trajectory07	random data test split	mean_all_trajectories
kr_model	od_model									
20kp FLOAT 224x224	FLOAT 288x384	0.72%	0.84%	0.65%	2.59%	1.13%	0.68%	3.53%	1.71%	1.45%
20kp QAT 224x224	QAT 288x384	1.91%	1.63%	2.17%	6.97%	1.83%	2.20%	10.05%	3.59%	3.82%
20kp EDGE 224x224	EDGE 288x384	2.09%	0.89%	1.16%	N/A	N/A	N/A	N/A	N/A	1.38%
20kp FLOAT 224x224	FLOAT 480x640	0.57%	0.57%	0.59%	1.85%	1.02%	0.38%	1.71%	1.04%	0.96%
20kp QAT 224x224	QAT 480x640	4.02%	2.82%	4.22%	3.20%	3.50%	4.38%	4.90%	4.15%	3.86%
20kp EDGE 224x224	EDGE 480x640	4.03%	2.19%	2.60%	100.00%	2.65%	3.46%	100.00%	N/A	30.70%
41kp FLOAT 224x224	FLOAT 480x640	0.46%	0.47%	0.70%	1.30%	0.50%	0.40%	1.33%	0.88%	0.74%

Relative Position Error [%] - 1 sigma (68.27%): Range restricted to [20; 300] [m]

		trajectory01	trajectory02	trajectory03	trajectory04	trajectory05	trajectory06	trajectory07	random data test split	
kr_model	od_model									
20kp FLOAT 224x224	FLOAT 288x384	0.72%	0.84%	0.65%	3.15%	3.58%	2.50%	36.69%	1.75%	
20kp QAT 224x224	QAT 288x384	1.91%	1.63%	2.17%	7.61%	100.00%	100.00%	100.00%	3.59%	
20kp EDGE 224x224	EDGE 288x384	2.09%	0.89%	1.16%	N/A	N/A	N/A	N/A	N/A	
20kp FLOAT 224x224	FLOAT 480x640	0.57%	0.57%	0.59%	2.08%	8.67%	8.46%	34.05%	1.06%	
20kp QAT 224x224	QAT 480x640	4.02%	2.82%	4.22%	3.76%	4.54%	5.25%	100.00%	4.18%	
20kp EDGE 224x224	EDGE 480x640	4.03%	2.19%	2.60%	100.00%	100.00%	100.00%	100.00%		
41kp FLOAT 224x224	FLOAT 480x640	0.46%	0.47%	0.70%	1.56%	7.78%	8.85%	41.28%	0.89%	

Rotation Error [deg] - 1 sigma (68.27%): Range restricted to [20; 100] [m]

		trajectory01	trajectory02	trajectory03	trajectory04	trajectory05	trajectory06	trajectory07	random data test split	mean_all_trajectories
kr_model	od_model									
20kp FLOAT 224x224	FLOAT 288x384	179.343	2.156	2.030	5.031	4.684	3.038	4.861	2.298	28.735
20kp QAT 224x224	QAT 288x384	177.759	18.896	3.394	12.877	178.755	5.213	6.942	4.271	57.691
20kp EDGE 224x224	EDGE 288x384	170.832	4.702	4.595	N/A	N/A	N/A	N/A	N/A	60.043
20kp FLOAT 224x224	FLOAT 480x640	179.595	1.804	1.309	4.085	4.238	2.685	3.387	1.608	28.157
20kp QAT 224x224	QAT 480x640	175.770	35.826	6.732	12.451	176.952	8.324	8.311	5.215	60.624
20kp EDGE 224x224	EDGE 480x640	174.959	6.198	178.983	178.415	10.604	7.882	179.046		105.155
41kp FLOAT 224x224	FLOAT 480x640	179.629	179.670	1.074	3.960	179.509	2.004	2.470	1.282	78.331

Rotation Error around X [deg] - 1 sigma (68.27%): Range restricted to [20; 100] [m]

		trajectory01	trajectory02	trajectory03	trajectory04	trajectory05	trajectory06	trajectory07	random data test split	mean_all_trajectories
kr_model	od_model									
20kp FLOAT 224x224	FLOAT 288x384	-177.174	0.456	0.366	1.326	-0.401	0.193	-0.373	0.444	-25.087
20kp QAT 224x224	QAT 288x384	-176.741	2.185	2.586	2.993	2.612	2.319	0.775	0.883	-23.324
20kp EDGE 224x224	EDGE 288x384	7.358	-1.944	0.142	N/A	N/A	N/A	N/A	N/A	1.852
20kp FLOAT 224x224	FLOAT 480x640	-178.213	0.243	0.201	1.803	-0.170	0.325	-0.606	0.333	-25.202
20kp QAT 224x224	QAT 480x640	-174.887	4.577	5.042	1.295	3.594	4.418	0.376	1.307	-22.226
20kp EDGE 224x224	EDGE 480x640	0.974	-0.382	1.040	176.888	-0.397	2.321	179.711		51.451
41kp FLOAT 224x224	FLOAT 480x640	177.913	179.272	0.370	0.309	178.919	0.346	0.290	0.247	76.774

Rotation Error around Y [deg] - 1 sigma (68.27%): Range restricted to [20; 100] [m]

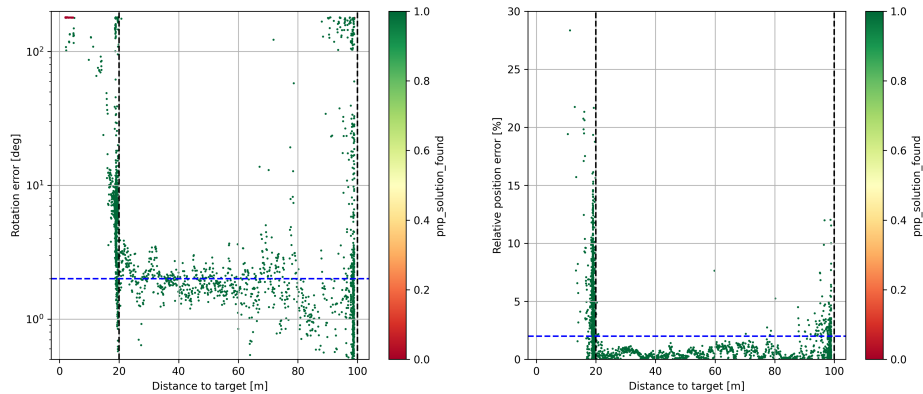
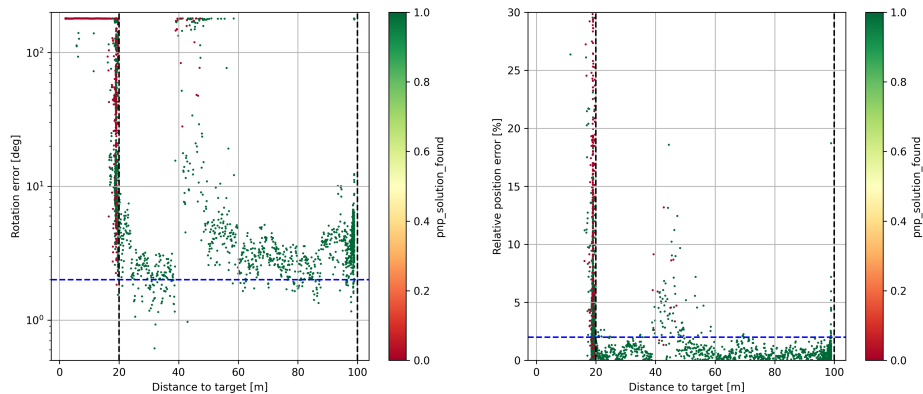
kr_model	od_model	trajectory01	trajectory02	trajectory03	trajectory04	trajectory05	trajectory06	trajectory07	random data test split	mean_all_trajectories
20kp FLOAT 224x224	FLOAT 288x384	-0.111	0.076	0.465	-0.337	-0.069	-0.341	0.400	0.408	0.012
20kp QAT 224x224	QAT 288x384	0.065	0.376	0.585	-1.017	0.689	-0.631	-0.092	0.599	-0.004
20kp EDGE 224x224	EDGE 288x384	0.768	-0.081	0.996	N/A	N/A	N/A	N/A	N/A	0.561
20kp FLOAT 224x224	FLOAT 480x640	-0.115	0.086	0.374	-0.675	-0.044	-0.348	0.273	0.272	-0.064
20kp QAT 224x224	QAT 480x640	-0.145	-0.072	0.232	-1.527	0.675	-0.628	-0.714	0.686	-0.311
20kp EDGE 224x224	EDGE 480x640	0.872	-0.114	-0.068	1.939	0.048	-0.645	0.455		0.355
41kp FLOAT 224x224	FLOAT 480x640	-0.054	0.033	0.381	-0.547	0.265	-0.241	0.412	0.226	0.036

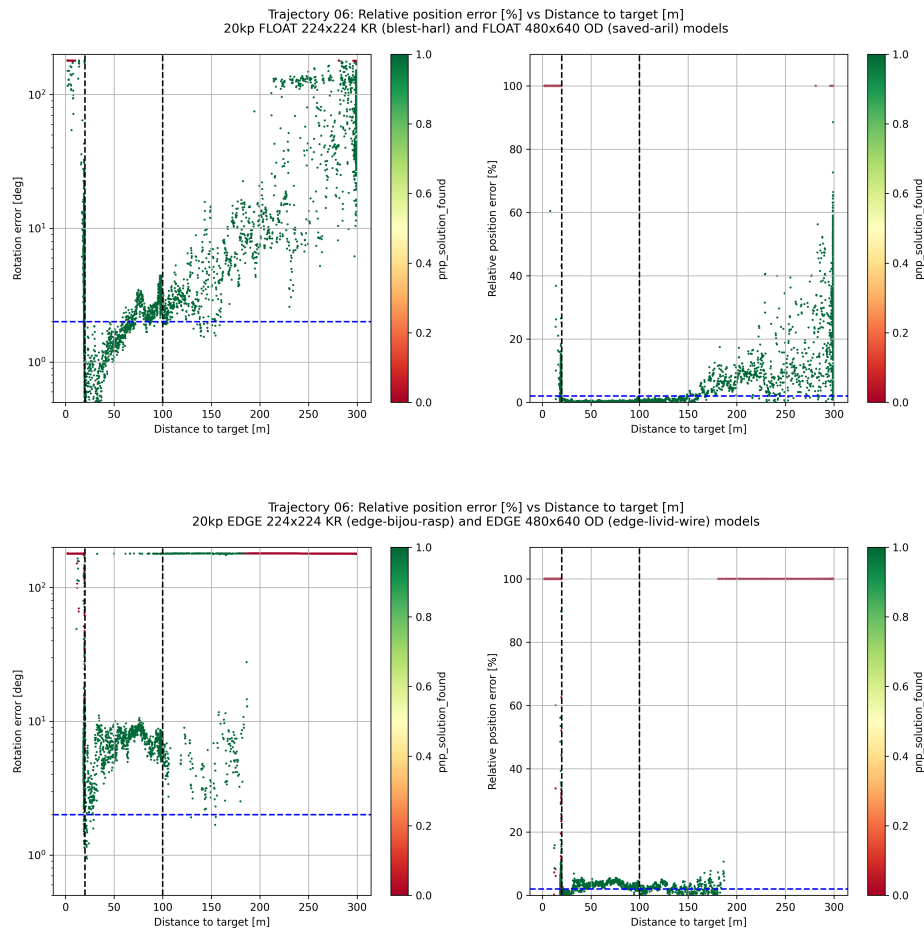
Rotation Error around Z [deg] - 1 sigma (68.27%): Range restricted to [20; 100] [m]

kr_model	od_model	trajectory01	trajectory02	trajectory03	trajectory04	trajectory05	trajectory06	trajectory07	random data test split	mean_all_trajectories
20kp FLOAT 224x224	FLOAT 288x384	-0.111	0.076	0.465	-0.337	-0.069	-0.341	0.400	0.408	0.012
20kp QAT 224x224	QAT 288x384	0.065	0.376	0.585	-1.017	0.689	-0.631	-0.092	0.599	-0.004
20kp EDGE 224x224	EDGE 288x384	0.768	-0.081	0.996	N/A	N/A	N/A	N/A	N/A	0.561
20kp FLOAT 224x224	FLOAT 480x640	-0.115	0.086	0.374	-0.675	-0.044	-0.348	0.273	0.272	-0.064
20kp QAT 224x224	QAT 480x640	-0.145	-0.072	0.232	-1.527	0.675	-0.628	-0.714	0.686	-0.311
20kp EDGE 224x224	EDGE 480x640	0.872	-0.114	-0.068	1.939	0.048	-0.645	0.455		0.355
41kp FLOAT 224x224	FLOAT 480x640	-0.054	0.033	0.381	-0.547	0.265	-0.241	0.412	0.226	0.036

Percentage of valid frames: Range restricted to [20; 100] [m]

kr_model	od_model	trajectory01	trajectory02	trajectory03	trajectory04	trajectory05	trajectory06	trajectory07	random data test split	mean_all_trajectories
20kp FLOAT 224x224	FLOAT 288x384	100.0%	98.8%	99.1%	100.0%	100.0%	100.0%	100.0%	99.5%	99.7%
20kp QAT 224x224	QAT 288x384	100.0%	98.6%	99.0%	99.7%	100.0%	100.0%	99.9%	99.0%	99.6%
20kp EDGE 224x224	EDGE 288x384	98.0%	96.9%	98.5%	N/A	N/A	N/A	N/A	N/A	97.8%
20kp FLOAT 224x224	FLOAT 480x640	100.0%	99.1%	99.2%	100.0%	100.0%	100.0%	100.0%	99.8%	99.8%
20kp QAT 224x224	QAT 480x640	100.0%	98.7%	99.2%	99.8%	100.0%	100.0%	99.9%	96.4%	99.7%
20kp EDGE 224x224	EDGE 480x640	88.7%	98.9%	98.7%	18.7%	100.0%	100.0%	14.2%		74.2%
41kp FLOAT 224x224	FLOAT 480x640	100.0%	99.0%	99.2%	100.0%	98.4%	100.0%	100.0%	99.7%	99.5%

Trajectory 02: Relative position error [%] vs Distance to target [m]
20kp FLOAT 224x224 KR (blest-harl) and FLOAT 288x384 OD (licit-weal) modelsTrajectory 02: Relative position error [%] vs Distance to target [m]
20kp EDGE 224x224 KR (edge-bijou-rasp) and EDGE 480x640 OD (edge-square-delf) models



8.5 Deep learning frameworks trade-off

The trade-off between PyTorch and TensorFlow (Keras) was initiated following the issues faced while embedding CNN models with Vitis-AI in DIOSSA phase 1. The main goal of the trade-off is to select the framework that is more likely to guarantee the successful deployment of CNN models with Vitis-AI. The evaluation of the two deep learning frameworks is included in the next table. The qualitative metrics were selected focusing on the target application, which is to be integrated in an edge device.

• API

- **PyTorch 1.x** : PyTorch includes a sequential container and a subclassing option. Both can be mixed and nested and **used with Vitis-AI without limitations** .
- **TensorFlow 2.x (Keras 2.x)** : Keras includes a sequential and a functional API, as well as a subclassing option. However, the Vitis-AI quantizer **only supports the functional API** , which itself **imposes some restrictions on the possible CNN models to be deployed** .

• Edge friendliness

- **PyTorch 1.x** : PyTorch had **PyTorch Mobile** (which has no footprint on edge deployment) , but it was replaced (in version 2.x) with a **new toolkit called ExecuTorch** (still to gain market traction) .
- **TensorFlow 2.x (Keras 2.x)** : TensorFlow has a significant lead in terms of edge deployment, with **TensorFlow Lite** , which is **widely adopted by the industry** .

• Off-the-shelf models availability

- **PyTorch 1.x** : PyTorch is more widely used in academic environments, which reflects on the **vast catalog of off-the-shelf models available** .
- **TensorFlow 2.x (Keras 2.x)** : The **catalog of off-the-shelf models in TensorFlow is smaller** , even more so if the single API supported by Vitis-AI is considered.

• Custom models development effort

- **PyTorch 1.x** : The Vitis-AI PyTorch **workflow requires many subclasses for each specific purpose (PTQ and QAT). All the models embedded were done so without facing operator or quantizer/compiler limitations** .
- **TensorFlow 2.x (Keras 2.x)** : The Vitis-AI TensorFlow **workflow requires the Keras functional API** , which has a more limited set of operations, but it **allows to fulfill all the needs with a single Python class** . However, the **limited set of operations combined with a more limited quantizer/compiler constitute a major roadblock** .

• Speed (training and inference)

- **PyTorch 1.x** : PyTorch **always runs in eager execution, being slower for training and inference** . (Version 2.x apparently allows easy compilation).
- **TensorFlow 2.x (Keras 2.x)** : TensorFlow **can easily switch between eager execution and compiled computational graph** , which **combines debugging easiness with significant speed advantage** .

TensorFlow is (in theory) the more edge friendly framework, and the well established TensorFlow Lite is the main proof, however the identified limitations in the Vitis-AI TensorFlow quantizer prevent the correct embedding of one of the selected CNN models and this issue was rated as showstopper. The Vitis-AI TensorFlow quantizer together with the required Keras functional API do not support a CNN model with shared layers, therefore PyTorch is the best and only option to successfully embed the CNN models required by the algorithms used in DIOSSA without having deviations from the original CNN models.

While PyTorch is slower (to train and infer) and less likely to be directly deployed to the edge, it has more off-the-shelf models publicly available. The nature of the CNN models (black box) make it very hard if not impossible to identify small issues in the CNN model definitions and the strategy adopted by LMO is to validate the results of the custom models against the off-the-shelf counterpart. Only after an exact match of the outputs for the same inputs is the development of a

custom model finished. This validation strategy is only possible if there is an off-the-shelf model available, so the large catalog of off-the-shelf models is a critical aspect.

The observations made in this trade-off are valid for Vitis-AI 3.5 (the latest version at the moment of writing). Newer versions of Vitis-AI might require to revise the trade-off, but unless required to move to a more recent version, for DIOSSA ultimately it is possible to freeze the version. The choice of CNN HW accelerator, the DPU in an UltraScale+ MPSoC, comes with a set of associated libraries and tools all under the umbrella of Vitis-AI, so the developments in DIOSSA are dependent on the supported features of Vitis-AI and the development roadmap of Vitis-AI can have some impact on DIOSSA.

An alternative to PyTorch or TensorFlow already available is Keras 3. With version 3, Keras will go back to its roots becoming framework agnostic once again (supporting TensorFlow, PyTorch and JAX), which means Keras will make it very simple to change the backend framework with minimal to no changes in source code. Being framework agnostic could potentially allow to eliminate the drawbacks of the backend frameworks by allowing to select the best framework for each target application. Unfortunately Vitis-AI does not support Keras 3 yet, so at least for the moment Keras 3 is not a possible option.

8.6 Catalog of models

The CNN models available in LMO's catalog are listed in the next tables, together with their capabilities. The PyTorch models are more developed than the Keras (TensorFlow) models, given the decision to halt the development of the Keras models based on the identified limitations of Vitis-AI TensorFlow quantizer.

Catalog of models for PyTorch

Function	Architecture	Inference	Training	PTQ	QAT	Compilation
Classification	ResNet	Yes	Yes	Yes	Yes	Yes
Classification	MobileNetV2	Yes	Yes	Yes	Yes	Yes
Object detection	Faster R-CNN	Yes	OTS	Yes	No	Yes
Object detection	FCOS	Yes	Yes	Yes	Yes	Yes

Keypoints regression	MobilePose	Yes	Yes	Yes	Yes	Yes
----------------------	------------	-----	-----	-----	-----	-----

Catalog of models for TensorFlow (Keras)

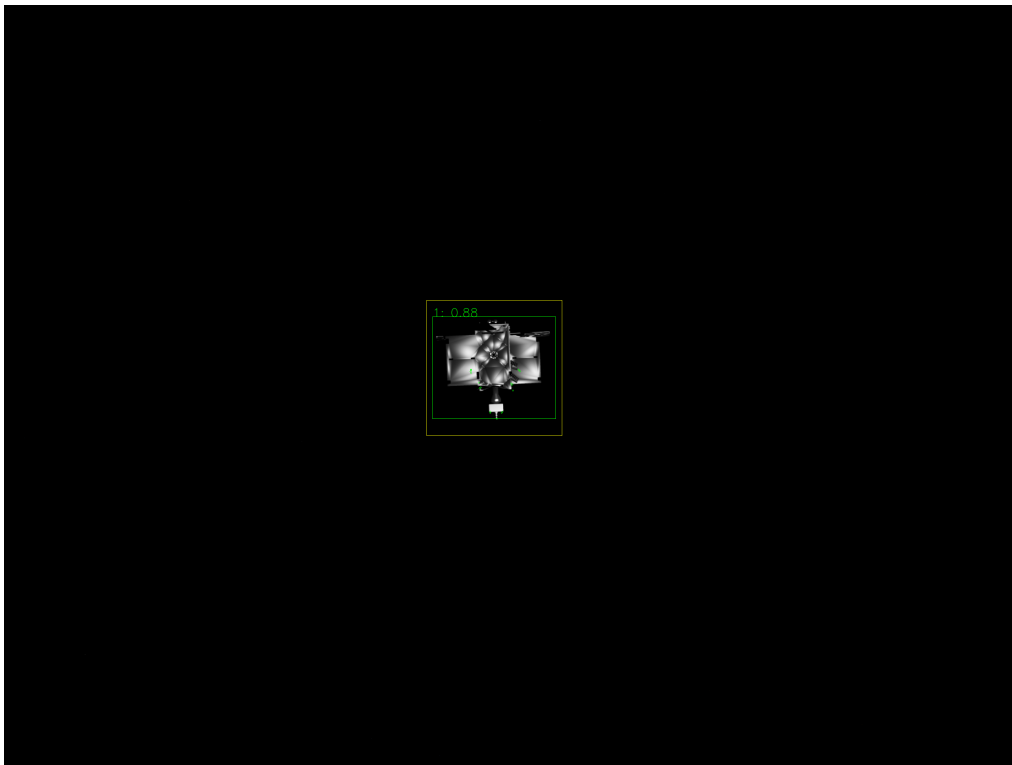
Function	Architecture	Inference	Training	PTQ	QAT	Compilation
Classification	ResNet	Yes	Yes	Yes	Yes	Yes
Classification	MobileNetV2	Yes	Yes	Yes	Yes	Yes
Object detection	Faster R-CNN	No	No	No	No	No
Object detection	FCOS	No	No	No	No	No
Keypoints regression	MobilePose	Yes	Yes	Yes	Yes	Yes

8.7 Embedding results

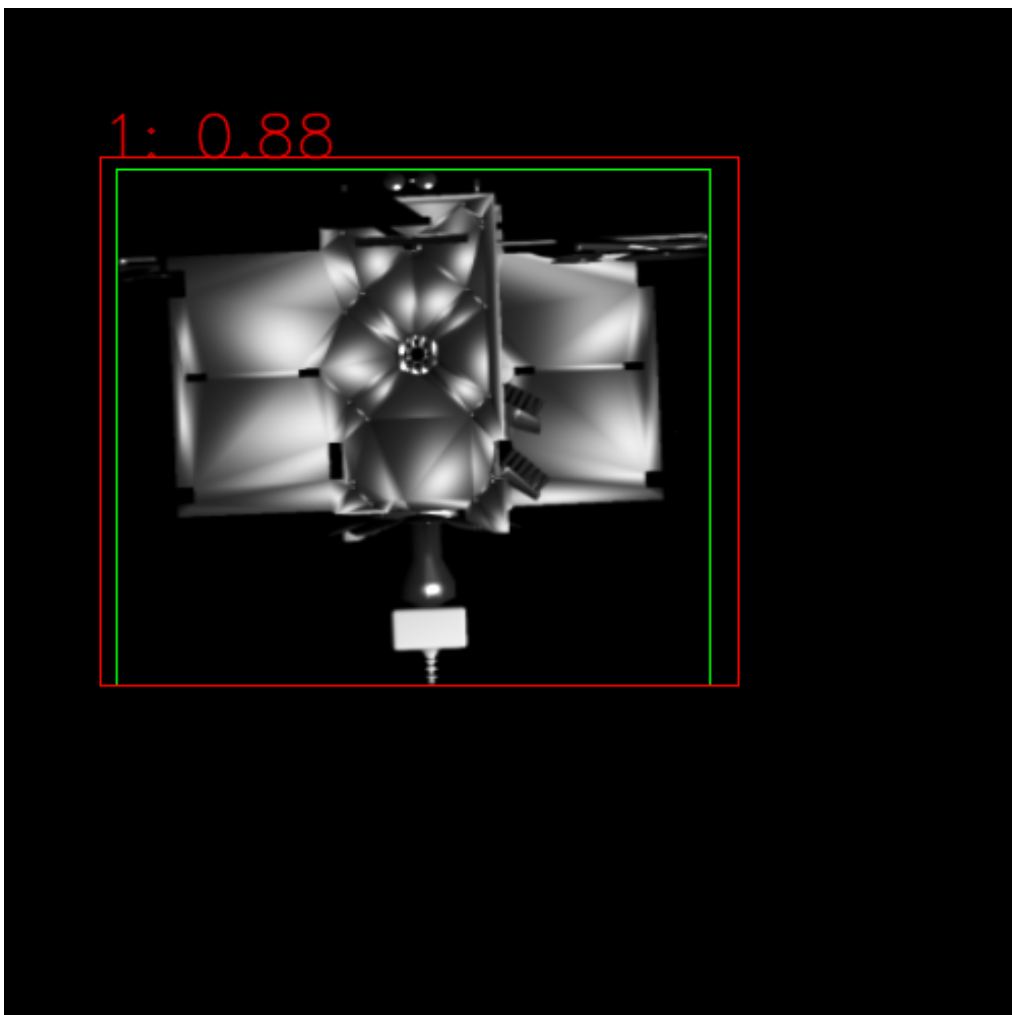
One of the identified major issues, during the first phase of DIOSSA, was the embedding aspect of the models. The initial concept of deploying directly the off-the-shelf models to the edge device, the DPU in the MPSoC, proved to be impossible in most cases, due to incompatible layers, present in the most CNN models, normally on the post-processing side of the models.

The action undertaken to overcome this roadblock was to develop custom models that besides enabling a successful deployment to the edge also support a quantization aware training process, which minimizes the accuracy gap between the float model and the compiled edge model.

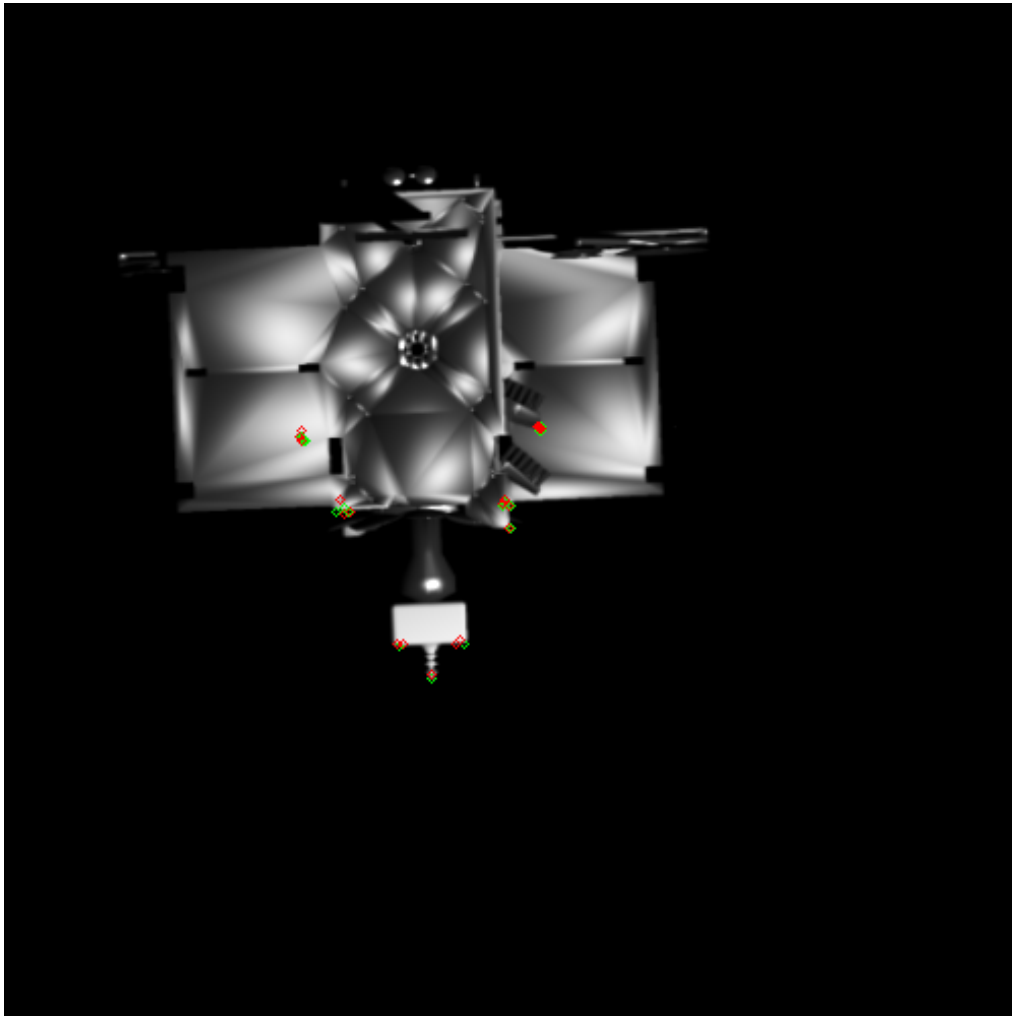
The outcome of this activity (custom models and quantization aware training) shows the CNN models work well on the edge device, validating the process. The following figures present in a qualitative way the comparison between the ground truth and the actual outputs of the CNN models from the edge hardware. It is possible to verify the results of the CNN models running on the edge hardware are close to the ground truth.



Synthetic image (at the original size) with the CNN's outputs overlaid in green.



Synthetic image (zoomed in) with the detection box overlaid in red and ground truth in green.



Synthetic image (zoomed in) with the keypoints overlaid in red and ground truth in green.

8.8 Sensor fusion trade-off

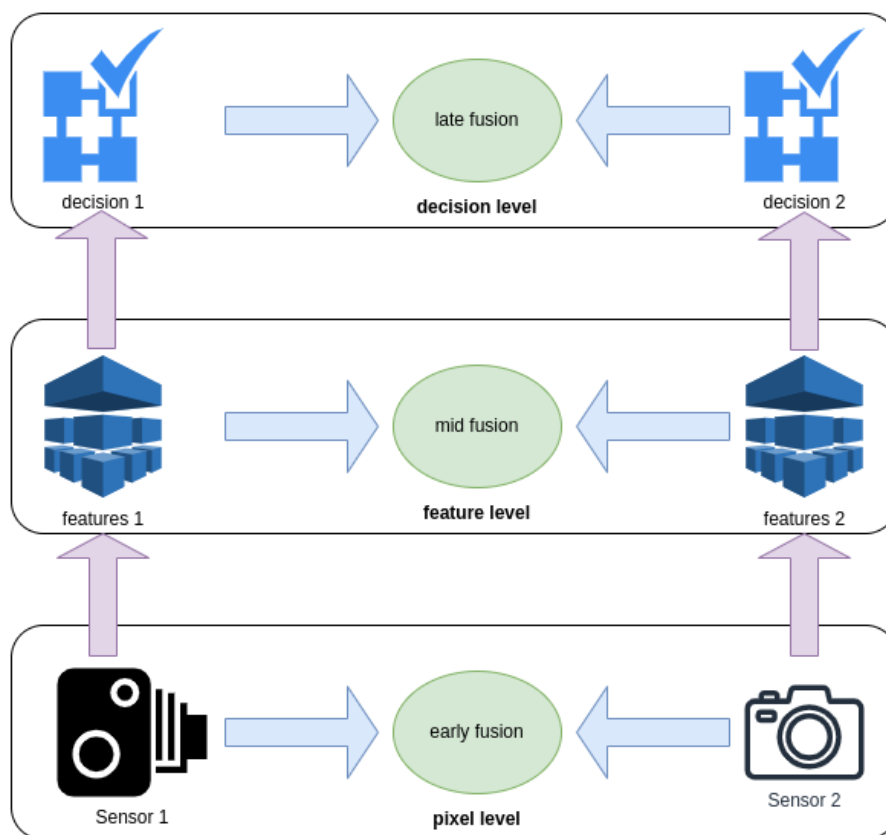
In the literature [RD-1] we can find several sensor fusion methods, which could be pretty well fit into three plus one categories:

1. Early fusion. The fusion happens at pixel-level (or any other raw data) and methods vary on how to combine different sensory outputs together and create a fused image, which will be fed to the downstream computer vision task.
2. Mid fusion. Here first features are extracted (using any classic feature descriptor or the output of a deep neural network feature layer) and fusion happens at this feature level by combining

those into a bigger feature vector, which will be fed to the remaining computer vision processing modules.

3. Late fusion. In this category fall methods, in which, each sensor is processed separately and provides its own decision (based on the task at hand, such as classification, keypoints detection, object tracking, etc.). The generated decisions are combined to form a final decision, following various approaches, most commonly using a voting scheme or interpolation.
4. Hybrid fusion. Finally the “plus one” method is a combination of early or mid with late fusion. Adding the late fusion can improve results by using information that might be lost during earlier stage fusion.

The figure below illustrates the three different architectures for sensor fusion.



Sensor fusion architectures

For the DIOSSA project, only visual and thermal infrared cameras are considered. For other applications fusion from other sensors could also be considered - Lidar, depth camera, event camera, or classical AOCS sensors such as accelerometers or star trackers.

While the visible camera collects more data thanks to its higher resolution, and potentially its 3-channels sensor for color cameras, the thermal infrared camera acquires the infrared radiation emitted by the target spacecraft as it generates heat from electronic units, making it usable in degraded light conditions, as for example in eclipse.

The same reasoning that led us to select a hybrid structure for the neural networks also applies for the sensor fusion: if both modalities (visible and infrared) are implemented separately up to decision making, it allows to better understand the performance of each of them and their limits. Late fusion also permits a more transparent decision by discarding information that would not reach a pre-defined confidence threshold.

Aside from performance outcome, which has not yet been evaluated, late fusion presents the drawback that the processing power hungry NN inference has to run twice in parallel. At this stage, it is considered to be acceptable, and if successful, optimization techniques will be investigated.

While other fusion techniques might provide interesting performance upgrade, they also involve more experimenting, and for the reasons stated above it has been decided to only implement and test late fusion for this phase of the project.

8.8.1 Late fusion implementation

In the selected late fusion approach, we have two processing pipelines, one for each sensor, as illustrated by the arrows pointing up in the figure above, producing two separate 3D pose estimations outputs.

The fusion is done at the decision level, meaning that for each timestamp, we select the estimated 3D pose from either the visible or the thermal sensor's output based on the *prediction_validity* flag. If both sensors provide a valid prediction, the result from the visible sensor is chosen. The final output will also include information indicating whether the result came from the visible or the thermal camera.

9 Simulator updates

9.1 Inertial measurement unit (IMU) model

In order to provide an accurate IMU measurement model, a noisy gyroscope and accelerometer model is introduced. The gyroscope model uses the angular rate of the spacecraft as ground truth where as the ground truth of the accelerometer model is chosen to be the acceleration of the spacecraft due to thruster firing. The gyroscope model contains the angular random walk (ARW), rate random walk (RRW) and bias instability (BI). Further, a bias can be set.

The mathematical model is

$$\omega_{meas} = \omega_{groundtruth} + \omega_{bias} + \omega_{arw} + \omega_{rrw}$$

For the accelerometer, the velocity random walk (VRW), acceleration random walk (AccRW) and bias instability are modelled. Also here, the a bias in the measurement can be set if desired.

The corresponding model is

$$a_{meas} = \frac{F_{groundtruth}}{m_{S/C}} + a_{bias} + a_{arw} + a_{rrw}$$

9.1.1 ARW and VRW model

The ARW and the VRW are modelled as a Gaussian white noise on the angular velocity and translational acceleration respectively. Integrating over time leads to a random walk in the angle or velocity.

9.1.2 RRW and AccRW model

The RRW and the AccRW are modelled as a Gaussian white noise on the angular acceleration and jerk respectively. Integrating over time leads to a random walk in the angular velocity or translational velocity.

9.1.3 Bias Instability

Modelling the bias instability is not as trivial as modelling a random walk. To model the bias instability a pink noise model is chosen.

Pink noise is modelled as a recursive IIR filter:

$$x_n = -a_1x_{n-1} - a_2x_{n-2} - a_3x_{n-3} - \dots + w_n$$

w_n represents a Gaussian white noise with a given standard deviation.

The filter coefficients are computed, using the following formula:

$$a_0 = 1$$

$$a_n = \frac{(n - 1 - \frac{\alpha}{2})a_{n-1}}{n}$$

Since the low frequency part of the Pink Noise model is of interest for the bias instability, the sampling frequency of the flicker noise generation is set to 1000 times the sampling frequency of the sensor. Thus, the noise value is held constant over this time.

To ensure a smooth signal, it is low pass filtered. Here, a first order filter low pass filter with a cutoff frequency of $cutoff = \frac{1}{\Delta t_{bias}}$

This gives the following filter coefficients

$$a_0 = 1 \quad a_1 = -0.99373647$$

$$b_0 = 0.00313176 \quad b_1 = 0.00313176$$

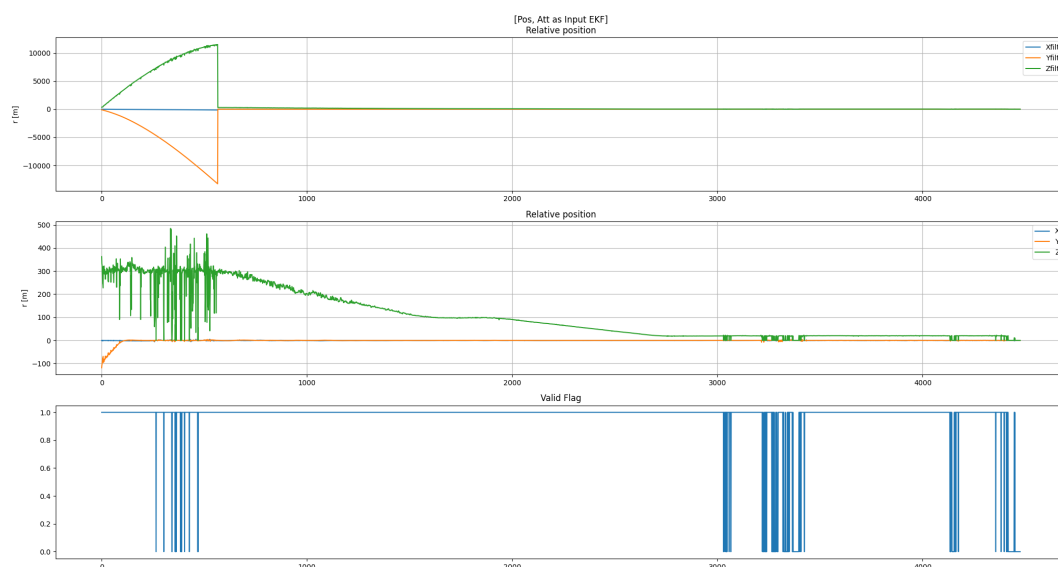
10 GNC flight software

10.1 Navigation filters

To ensure a consistent and more reliable measurement process, the output of the neural networks is integrated into a navigation filter. During an earlier phase of this study, two Extended Kalman Filters (EKFs) were developed, each differing in terms of the required measurement inputs and their underlying measurement models.

To evaluate the performance of these filters with new models, synthetic images were introduced, generating either detected keypoints or estimated pose as inputs for the filters.

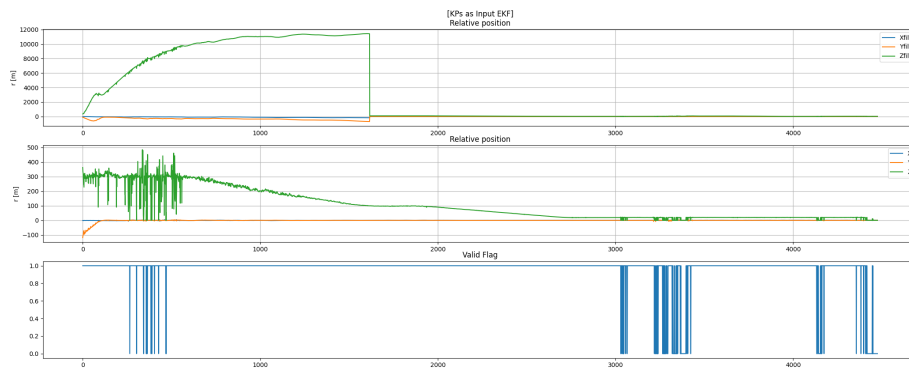
The graph below illustrates the relative position estimated by the EKF using quaternion and position data as inputs from the models.



Bad visibility pose as input

The results show that the filter diverges for the initial measurements. Additionally, the output of the models contains a significant number of outliers, leading to non-Gaussian measurement noise.

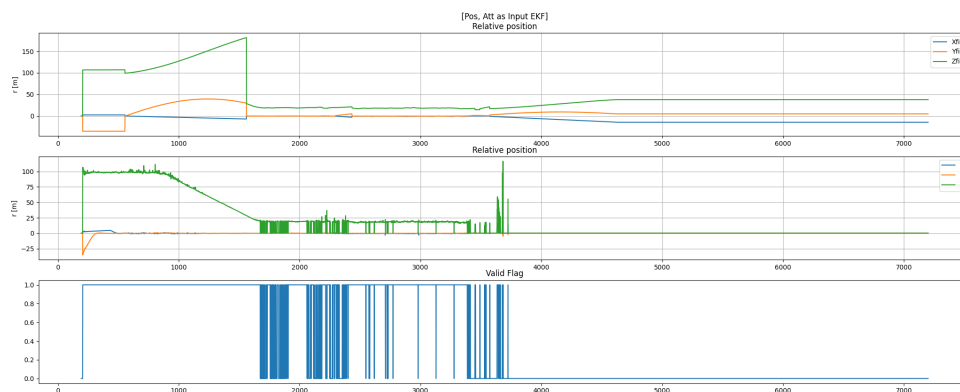
Using the EKF variant that processes keypoints as input yields the following results:



Bad visibility KPs as input

Similar behavior is observed in this scenario. It is important to note that the reduced visibility of the target satellite in this case causes a notable increase in measurement noise. For comparison, a scenario with improved visibility was analyzed.

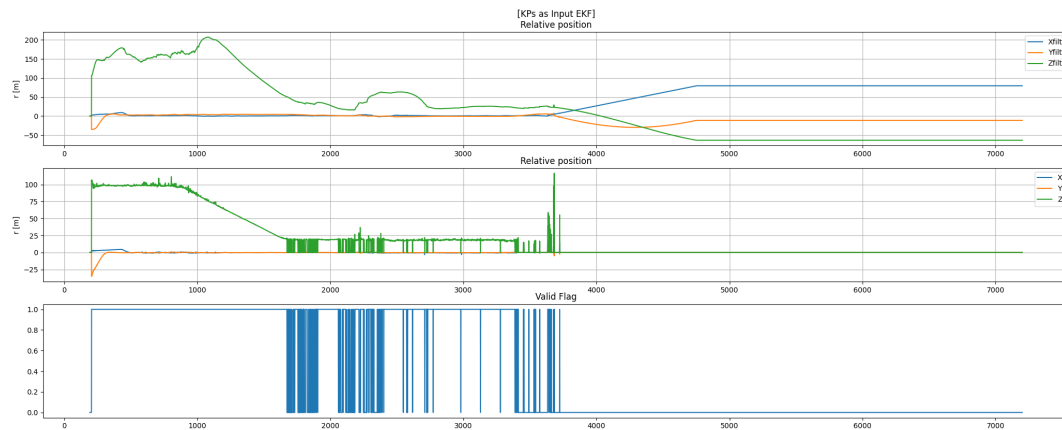
The following graph shows the estimated relative position using an EKF with pose data as input:



Good visibility pose as input

In this case, the measurement noise is closer to Gaussian. However, the approximation remains insufficiently accurate, causing the filter to diverge.

Further, the version taking keypoints as input is investigated.



Good visibility KPs as input

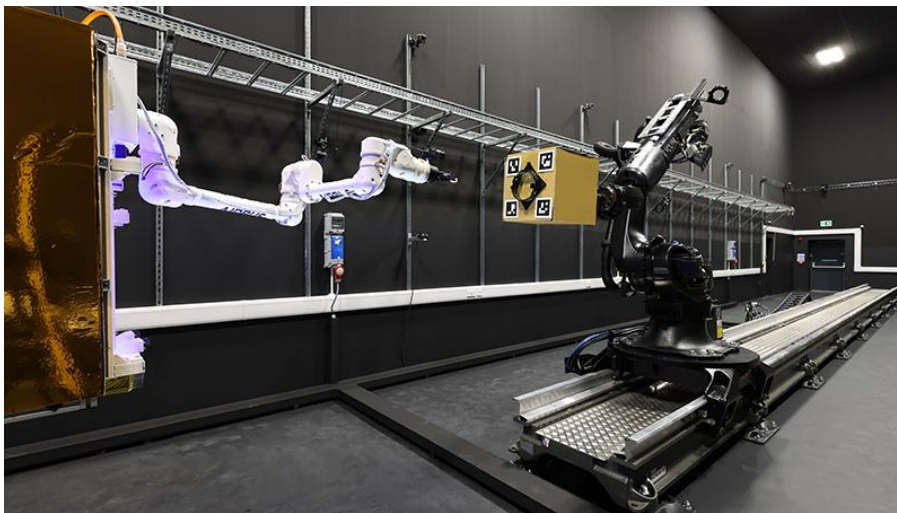
Here, the filter demonstrates improved convergence behavior but still exhibits slight divergence at the beginning.

In conclusion, the current filters, which assume Gaussian measurement noise, lack robustness against outliers, resulting in non-Gaussian noise. To address this limitation, a filter capable of handling non-Gaussian noise will be developed in the next phase.

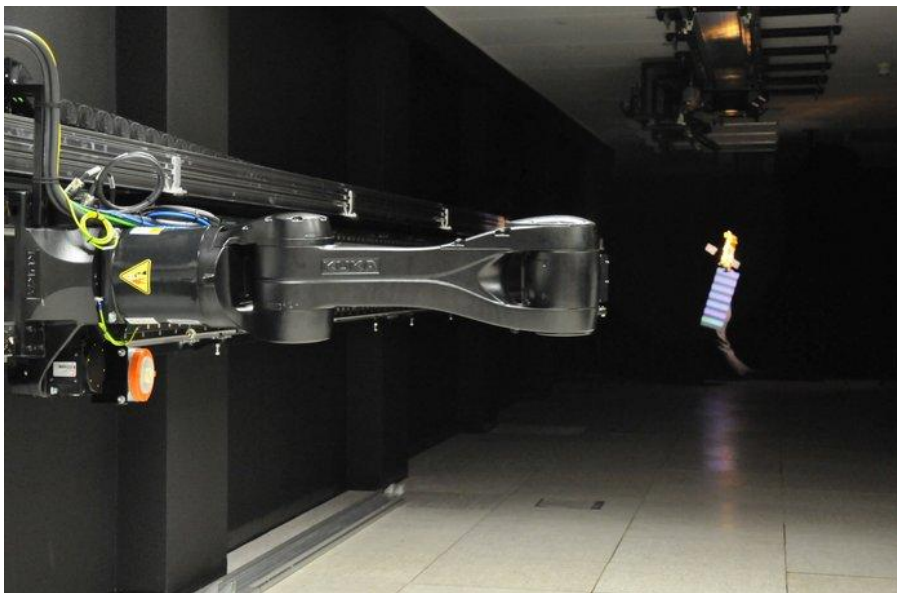
11 Functional testing facility

For the functional testing of the RPO system, two facilities have been identified:

1. The IOSM Yard in Westcott (UK).
2. The GNC Rendezvous, Approach and Landing Simulator (GRALS) located at ESTEC (NL).



IOSM Yard



GRALS testbed

While the two facilities provide similar capability of simulating approach trajectory with two robotic arms in a dark environment, their visit highlighted some differentiating factors summarized in the table below.

Lab characteristics

Criteria	IOSM Yard	GRALS
Useful length for serving satellite (camera)	17 m	28 m
Payload capacity	Up to 120 kg	Up to 10 kg
Light source	Fixed on tripod, moved manually	Installed on 3rd robotic arm, automated
Control system	ROS, outputs in rosbags	Matlab/simulink, outputs in text files
Operations	From control room on second floor	In the same room, protected by laser barrier

While the greater payload capacity (of the IOSM lab) would make the mockup design easier, the other points (of the GRALS lab) would make the rest of the testing activities (preparation, execution, data processing, etc.) more flexible. Therefore, it has been decided to select the GRALS facility for the functional testing activities of this development phase.

12 Functional testing satellite model

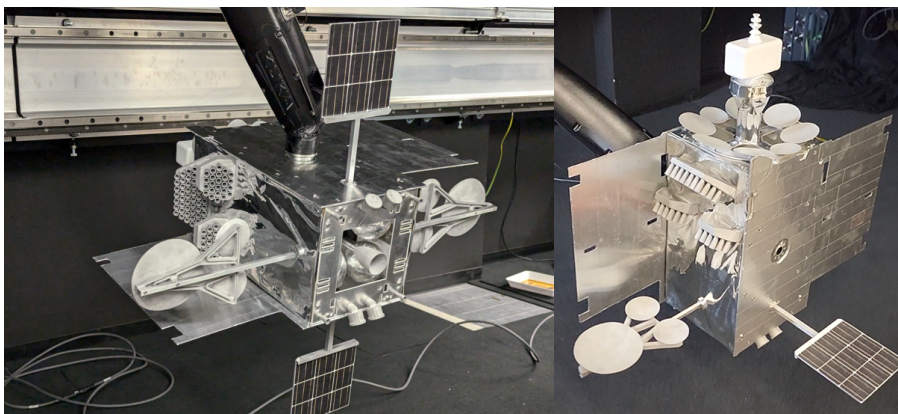
For the functional testing of the RPO system, we have identified and modeled a telecommunications satellite located on a GEO orbit: Inmarsat-5 F4.



Artist view of Inmarsat-5 F4

Existing 3D satellite models were utilized to create a CAD model and drawings that conform to the lab's constraints: - Maximum weight of 10kg on the robotic arms - Under 6kg there aren't any load factor issues with the robotic arms

The Inmarsat-5 F4's body (without the solar panels) is dimensioned as such : **3.27m x 6.98m x 8.08m** (Length x Height x Width). To fit the model into the lab and on the robotic arm, a scaling factor of **12** has been implemented on the design. The model main body is now dimensioned as follows: **0.2725m x 0.582m x 0.674m** (Length x Height x Width).

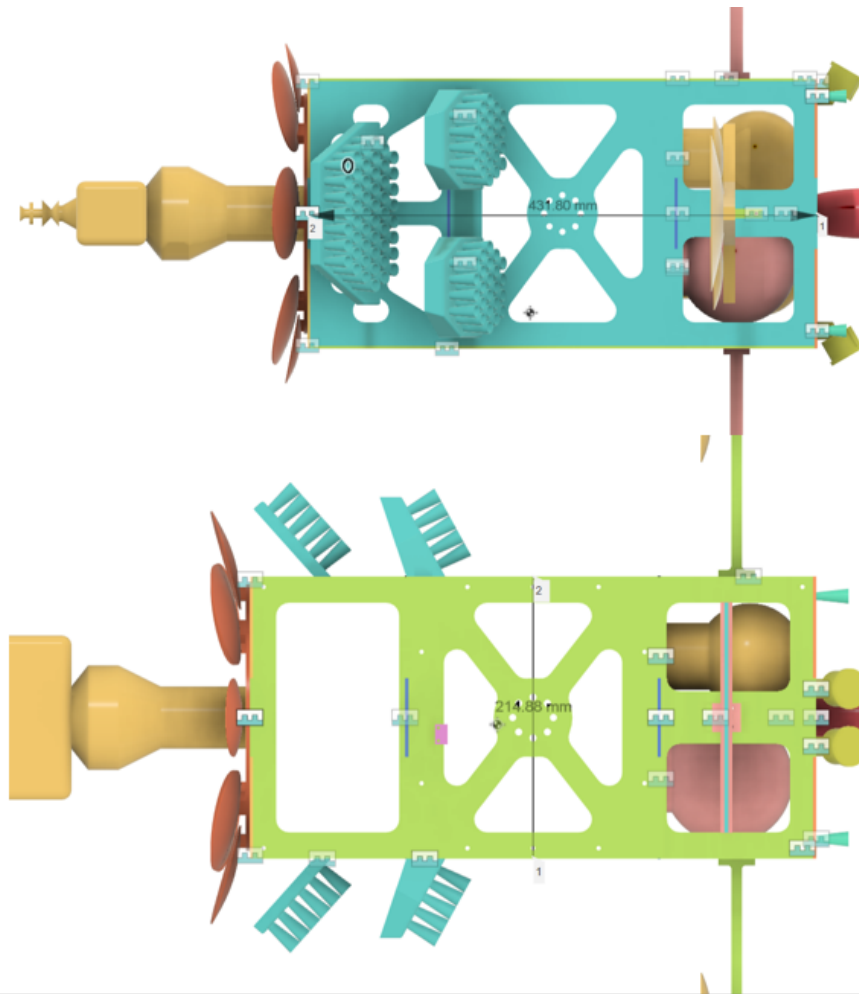


Manufactured and Assembled Mockup

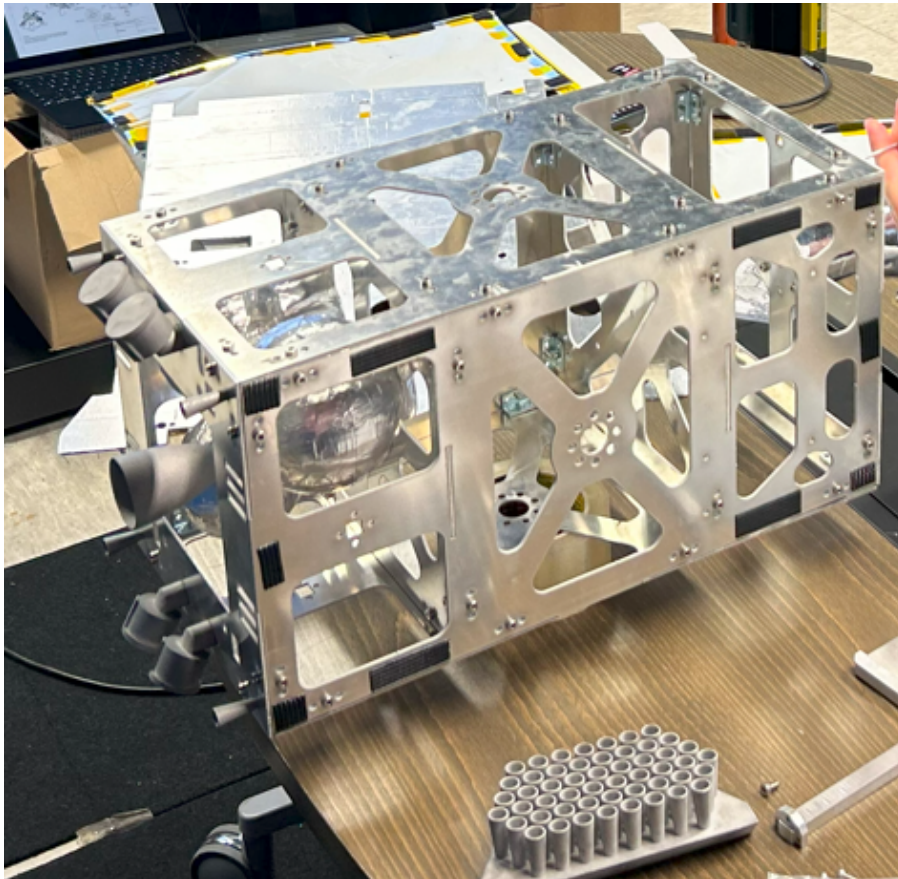
The factor of 12 provides an effective balance between weight and feature precision in manufacturing. Weighing just 7.3 kg, the mockup does not impose significant weight limitations on the GRALS facility's robotic arms. The selected materials contribute to the mockup's lightweight nature. Each material listed below helps achieve this reduced weight:

- machined aluminium, used on structural parts and outer panels,
- 3D printed aluminium, used for features and antenna dishes,
- aluminium tape, applied on the panels to give a shinier optical feature,
- white matte plastic, used for the transmitter at the end of the main body (opposite to the large apogee engine),
- mylar, used on the uncovered parts of the satellite main body,
- printed stickers, used to simulate the solar panels.

Along with the material selection, the mockup's design features areas where material has been removed, primarily from the structural panels that form the framework of the main body. These structural panels help maintain the mockup's structural integrity when assembled while keeping its weight minimal.



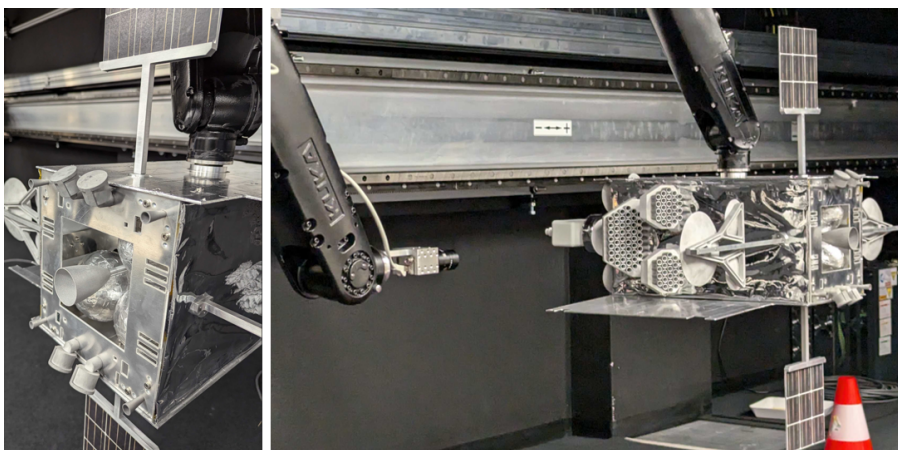
Mockup Structural Panels



Main Body Structure Under Assembly

The solar arrays have been significantly shortened, as the satellite's total length under space operation conditions is 33.8 meters. As a result, even at a 1/12 scale, the solar panels would either not fit in the GRALS facility or would impose additional constraints on the robotic arms and the testing campaign.

Although the mechanical interface with the robotic arm can be placed in various locations on the mockup, the ideal position is on the face of the satellite body where the solar panels are mounted. In this position, the camera has an improved view of the key features of the satellite model on both sides of its body.



Mockup-Robotic Arm Mechanical Interface

